Part II: Index

Symbols

- 63
-- 69
^ 78, 83
^= 79
_ 36, 1107
, 1129
; 101
: 159, 342, 1104, 1291
:: 726
! 78, 1105
!= 75, 428
? 582, 584, 590
? extends T 583, 591
? super T 584, 591
?: 90
. 9, 127, 128, 328, 340, 1129, 1292
.. 1293
... 137, 141
' 37, 38
" 39
""" 460
[] 92, 118, 121
{} 101, 120, 356
@ 1562, 1567
@Deprecated 1584
@Documented 1578
@FunctionalInterface 1583
@Inherited 1578
@Override 200, 226, 243, 620, 1582
@param 100, 112
@Repeatable 1579

@Retention 1571
@return 186
@SafeVarargs 634, 652, 661, 865, 1589
@SuppressWarnings 1586
@SuppressWarnings("unchecked") 629, 634
@Target 1573
@throws 392
* 58, 61, 332
*= 66
/ 61, 62, 1291
/* and */ 40
/** and */ 41
// 40
/= 66
\ 38, 1105, 1291
& 78, 82
&& 80
&= 79
1105, 1129, 1150
% 61, 62
%= 66
+ 63, 67
+ concatenation 450
++ 69
+= 66
< 74, 1150
<< 86, 87
<<= 90
<= 74
<> 569, 572, 648
-= 66
= 54, 1104
== 75, 428, 435
-> 92, 162, 681
> 74
>= 74
>> 86, 88
>>= 90
>>> 86, 89
>>>= 90

| 78, 82, 401, 1150
|= 79
|| 80
~ 82

Numerics

0 1129

A

ability interfaces
see marker interfaces
abrupt method completion 369
absolute adjusters 1039
absolute path 1292
absolute values 480
abstract
interfaces 241
methods 242, 677
abstract 241, 242, 677
abstract classes 220
abstract methods 226
considered incomplete 224
declaring 221
extending 222
incomplete 221
polymorphic references 222
UML class diagram 221
abstract data type 567
abstract keyword 221, 226
abstract method declarations 677
in enum types 296
in interfaces 242
abstract methods 226
declaration 226
implementing 226
method header 226
overloading 227
overriding 226

abstraction 5, 13
access modifiers 99, 100, 109, 347
access order 843, 844
accessibility 10, 19, 354, 1177
    access modifiers 347
    anonymous classes 527
    local classes 514, 515
    members 349, 354
    modifiers 495
    non-static member classes 506
    static member classes 500
    top-level types 347
    UML notation 350
accessibility for class members
    package 349, 353
    private 353
    protected 352
    public 352
accessibility for top-level types
    package 347
    public 347
accessibility rules 1178
accessor methods *see* **get methods**
accounting currency formatting 1122
acquisition order 1465
activation frame 536
    see method execution 367
actual parameter 128
actual parameter list 127, 267
actual type parameters 571
adapter 994, 995, 996
adding to class 193
additive operators 63
aggregate module 1172
aggregate of values 302
aggregate operations 883
aggregation 13, 16, 196
    hierarchy 197
algorithms 785
aliases 55, 76, 77, 131

see also **symbolic link**
all-or-none model 1478
ambiguous call 204, 268
analyzing program code 1624
and operator 83
AND-ing 86
AnnotatedElement 1592
Annotation 1561
annotation 1561
@ 1567
applying annotation type 1567
declaration annotation 1575
default values 1569
marker annotation 1567
metadata 1561
normal annotation 1567
processing 1591
single-element annotation 1567
type annotations 1576
annotation type 1561, 1562
@ 1562
access modifier 1563
annotation type element 1564
containing annotation type 1580
declaring 1562
marker annotation type 1564
member declarations 1563
multi-element annotation type 1565
repeatable annotation type 1580
simple name 1563
single-element annotation type 1564
syntax 1562
annotation type declaration
see **annotation type**
annotation type element 1564
access modifier 1565
attributes 1564
default values 1566
element type 1565
method declaration 1564

optional annotation element 1566
required annotation element 1566
`value()` element 1564, 1565
annotations 241
`@Deprecated` 1584
`@FunctionalInterface` 677, 1583
`@Override` 200, 620
`@SafeVarargs` 634
`@SuppressWarnings("unchecked")` 629
anonymous arrays 122, 126
`[]` 122
anonymous classes 496, 523, 573, 692
accessing members in enclosing context 527
compared with lambda expressions 694
declaration 523
event listeners 523
extending an existing class 523
hiding members in enclosing context 527
implementing an interface 525
in non-static context 523
in static context 523
instantiation 526
objects on the fly 523
referencing instances 526
shadowing local variables 527, 694
using qualified `this` 527, 695
anti-symmetric 764
API (application programming interface) 2
apostrophe 38
application 19
architecture neutral 4
argument
see **actual parameter**
arguments to main method 141
arithmetic compound assignment
operators 66
`ArithmeticException` 374
arity 136, 697
array constructor references 727, 733
array creation expression 119, 122, 144

array element initializer 1568
array initializer 120, 123, 126, 146
array list 804
array mismatch 873
array operations 800
array store check 262, 629
array store check report
    false positive 633
array types 581
    see **arrays**
ArrayBlockingQueue 1500
ArrayDeque 825
ArrayIndexOutOfBoundsException 121, 374, 399
ArrayList 452, 804
    add collection 653
    add element 649, 653
    autoboxing 656
    capacity 648
    clear list 654
    comparison with arrays 664
    constructing 648
    constructors 650
    converting to array 660
    element search 658
    filtering 690
    import 648
    inheritance hierarchy 647
    initial capacity 649
    insertion order 646
    list of lists 650
    membership test 657
    modifying 653
    nested lists 650
    object value equality 658
    open range-view operations 647
    ordered 646
    position-based access 647
    positional index 657
    positional order 646
    positional retrieve 657

querying 657
references 648
remove collection 654
remove element 654
replace element 654
size 648, 657
subtype relationship 650
text representation 649
trim to size 655
type-safety 648, 650
unchecked conversion warning 648
zero-based index 646
Arrays 870
creating streams 900
arrays 118, 427
[] 118, 119, 121
{} 120
anonymous 122, 126
array creation expression 119
array initialize list 120, 123
array initializer 120, 126
array name 118
array size 119
array store check 262, 629
ArrayIndexOutOfBoundsException 121
binary search 868
bounds 121
collection operations 800
common prefix 872, 873
comparators 869
comparing 870
construction 119
converting to list 661
declarations 118
default initialization 119
dynamic 647, 804
element access expression 121
element default value 262
element type 118
elements 118, 121

equality 870
filling 875
index 118
index expression 121
initialization 119, 120, 125
initializing 875
iterating over 178
iterating over an array 121
length 118
lexicographic comparison 871
mismatch 873
multidimensional 122, 124
objects 427
proper prefix 873, 874
ragged 125
reference 118, 121, 262
searching 868
sequential streams 900
sorting 867
subtype covariance 261
text representation 875
using 121
arrays of arrays 118, 124
multidimensional 124
`ArrayStoreException` 262, 374, 650, 660, 800, 801
array-valued single-element annotation 1570
arrow notation (`->`) 162
arrow operator (`->`) 92
ascending-order key set 849
ASCII 37, 42
`AssertionError` 376
assessor methods
see **get methods**
assignable 47, 266
assignment
widening reference 586
assignment compatible 47, 266, 648
assignment conversions 47
assignment operator 8
assignments

arithmetic compound operators 66
bitwise 85
cascading 55
compound operators 79, 85
expression statement 55
extended operator 90
implicit narrowing 56
multiple 55
numeric conversions 56
operator 50, 54
primitive values 54
references 55
widening reference conversions 198
association 16
aggregation 196
composition 197
realization 246
associativity 51, 52
asynchronous 1374
atomic actions 1459, 1460, 1473
CAS: Compare and Set 1461
atomic classes 1461
atomic move 1312
atomic operation 1309, 1343
atomic variables 1460
atomic actions 1460
`AtomicInteger` 1461, 1462
atomicity 1459
`AtomicMoveNotSupportedException` 1317
at-sign 1562
attributes 1516
attributes *see* **properties**
`AttributeView` interface 1338
autoboxing 432
`for(:)` statement 179
see **boxing conversion**
`AutoCloseable` 411, 414, 538, 1238, 1248, 1523
streams 891
automatic garbage collection 535
automatic memory management 535

automatic module 1210, 1211
naming 1211
Automatic Resource Management (ARM) 413
automatic variables *see* **local variables**
auxiliary methods 254
available time zones 1077

B

backing collection 858
backslash 38
backspace 38
backtrace 407
backward compatibility 567
bags 806
balanced tree 790, 791, 814
barrier
see **cyclic barrier**
barrier action 1475
barrier point 1475
base 33, 433, 437
base case 1452
base class 193
base name 1106
basic collection operations 792
basic `for` statement 176
basic map operations 833
basic statement 1527
`BasicFileAttributes` interface 1334
`BasicFileAttributeView` interface 1338
Before Current Era (BCE) 1033
behavior 677
behavior parameterization 677
behavioral parameters 911
`BiConsumer` functional interface 711
primitive type specializations 713
`BiFunction` interface 719, 720
composing two-arity functions 720
primitive type specializations 721
big numbers 486, 487

`BigDecimal`
customizing formatting **1129**
formatting **1121**
parsing **1123**
rounding **1126**
`BigDecimal` class **487**
`BigInteger` **487**
binary
numeric promotion **50**
operators **50**
shift operators **86**
binary files **1242**, **1243**, **1244**
binary operator **52**
binary representation
see **binary values**
binary search
arrays **868**
lists **863**
binary values **1242**, **1243**, **1244**
`BinaryOperator` interface **723**, **724**
composing binary operators **723**
primitive type specializations **724**
bind parameter **1530**
bind variable **1530**
binding of method call **207**, **229**
`BiPredicate` functional interface **705**
bit mask **84**
bit masking **86**
bit patterns **54**
bit shifting **86**
bitwise
and operator **83**
assignment **85**
complement **83**
compound assignment **85**
operators **82**
or operator **83**
shifting **86**
xor **83**

bitwise AND
`&` 82
bitwise complement
`~` 82
bitwise exclusive OR
`^` 83
bitwise OR
`|` 82
bitwise XOR
`^` 83
blank `final` local variables 234
blank `final` variables 136, 231, 233
block 163, 168, 171
block scope 171, 357, 686
`BLOCKED` state 1386
blocking collection 1487
blocking operations 1238, 1409
blocking queues 1499
`BlockingDeque` 1500, 1502
`BlockingQueue` 1500, 1501
blocks 101, 356
scope 356
`try` 379
boilerplate code 411, 693
`Boolean`
condition 154
`Boolean` wrapper class 439
booleans 41, 43
casting 49
expressions 74
literals 37
`BooleanSupplier` functional interface 701
bounded collection 1487
bounded instance method references 729
bounded type parameters
constraint 593
bounded wildcard 583
bounds
constraint 593
boxing conversions 45

`break` statement 159, 182
bridge method 617, 766
BS *see* **backspace**
buckets 755
buffered readers 1255
    creating streams 1256
    end of stream 1256
    reading text lines 1256
buffered writers 1254
`BufferedReader` class 1246, 1319
    streams 904
`BufferedWriter` class 1247, 1320
buffering streams 1242
building abstractions 13
built-in functional interfaces
    `java.util.function` package 697
    `BiConsumer` 711
    `BiFunction` 719, 720
    `BinaryOperator` 723, 724
    `BiPredicate` 705
    `BooleanSupplier` 701
    `Consumer` 711
    `DoubleBinaryOperator` 724
    `DoubleConsumer` 711
    `DoubleFunction` 714
    `DoublePredicate` 705
    `DoubleSupplier` 701
    `DoubleToIntFunction` 715
    `DoubleToLongFunction` 715
    `DoubleUnaryOperator` 722
    `Function` 714
    `IntBinaryOperator` 724
    `IntConsumer` 711
    `IntFunction` 714
    `IntPredicate` 705
    `IntSupplier` 701
    `IntToDoubleFunction` 715
    `IntToLongFunction` 714
    `IntUnaryOperator` 722
    `LongBinaryOperator` 724

LongConsumer 711
LongFunction 714
LongPredicate 705
LongSupplier 701
LongToDoubleFunction 715
LongToIntFunction 715
LongUnaryOperator 722
ObjDoubleConsumer 711
ObjIntConsumer 711
ObjLongConsumer 711
Predicate 705
Supplier 701
ToDoubleBiFunction 720
ToDoubleFunction 714
ToIntBiFunction 720
ToIntFunction 714
ToLongBiFunction 720
ToLongFunction 714
UnaryOperator 722
built-in synchronization
see **intrinsic locking**
bulk map operations 841
bulk operations on collections 792
bulk task execution 1443
bundling resources 1106
byte 32, 42
byte streams 1237
reading bytes 1238
writing bytes 1238
bytecode 4, 21
bytecode verification 1615

C

C++ 4
cached thread pool 1430
caching 360
call by reference 132
call by value 132
call chaining 128

call signature 268
call stack
see **JVM stack** 367
callable statement 1534
stored functions 1534
stored procedures 1534
callee 127
caller 127, 186
candidate bundle name 1112
capacity 648
capture conversion 608
capture of 607
carriage return 38, 39
CAS: Compare and Set 1461
cascaded grouping 989
cascaded partitioning 993
cascading assignments 55
cascading `if` - `else` statements 157
`case` constants 158, 159, 162
`case` labels 158, 159, 162
case sensitivity 30
case-insensitive ordering 773
cast 633
references 283
cast operator 45, 48, 50, 57, 66, 67, 76, 143, 271
casting 47, 48
see also **conversions**
`catch` clause 379
multi- 399
rethrowing exception 403
uni- 378
catching exceptions 367
catch-or-declare rule 390
catch-or-specify requirement 390
cause 407
ceiling entry 848
ceiling key 848
chaining 557
constructors 215, 557
`new` operator 509

chaining exceptions 407
changing list contents 865
`char` 42
character case 450
character encoding 1245
    see also **character sets**
character sequences
    see **strings** *and* **string builders**
character set
    ASCII 37, 42, 1245
    ISO Latin-1 37, 42
    Unicode 37, 42, 1245
character streams 1237, 1245
    character encoding 1245
    end of stream 1247
    reading characters 1247
    writing characters 1248
`Character` wrapper class 439
character-based device
    console 1260
characters 42
    literals 37
    searching for 453, 468
`CharSequence` interface 446, 452, 903
charsets
    see also **character sets**
checked exceptions 376
child class 193
child threads 1377, 1382, 1458
choice format 1150, 1152
    limits 1150
    subformat pattern 1150
    syntax 1150
`ChoiceFormat` 1119, 1152
choosing between `String` and `StringBuilder` class 466
`ChronoField` enum type 1040, 1050, 1083
    list of enum constants 1050
`ChronoUnit` enum type 1044, 1045, 1048, 1085, 1086
    list of enum constants 1048
`Class` 1592

`Class` class 428
class design 1615
class file 21
class file name 499
class hierarchy
    see **inheritance hierarchy**
class initialization 232
class literals 638
class lock 1384, 1395, 1396
class method 13
class path 339, 1210
    absolute pathnames 342
    entries order 342
    entry-separator character 342
    fully qualified package name 342
    path-separator character 342
    relative pathnames 342
    search in a named package 341
    searching for classes 339
    whitespace 342
class search path
    see **class path**
class variable 13
class versioning 1277
`ClassCastException` 273, 374, 1109
classes
    `abstract` 220
    accessibility 347
    adding to 193
    anonymous 494, 496, 523
    base 193
    body 99
    child 193
    class file name 499
    concrete 220, 222, 228
    constructors 109
    contract 220, 244
    declaration context 495
    declarations 99, 327
    definitions 6, 8

derived 193
diagram 11, 12
encapsulation 326
extending 193, 227
`final` 227, 360
`final` vs. `abstract` 228
fully qualified name 339
fully qualified package name 330
generalized 196
grouping 328
header 99
immutable 358
implementation 220
implementing interfaces 242
incomplete 221
initialization 232, 559
inner *see* **inner classes**
instances 8
local 494, 496, 514
members 9
methods 115
name 328
nested *see* **nested classes**
non-static member 494, 496, 503
`Object` 427
on-the-fly 494
parent 193
partial implementation 220
records 301
runtime 428
scope 354
sealed 314
searching for 339
specialized 196
static fields 113
static members 113, 496, 497
static methods 115
static variables 113
subclass 13, 193
superclass 13, 193

top-level 493, 496
utility 115
wrappers 427, 431
class-level lock
see **class lock**
`ClassLoader` class 427
`ClassNotFoundException` 373
`CLASSPATH` environment variable
see **class path**
`-classpath` option
see **class path**
clauses
`catch` 379
`extends` 193
`finally` 379, 384
`implements` 242
`permits` 314
`throws` 390
cleaning up 538
clean-up code 384, 1382
client 10
`Cloneable` interface 429
`CloneNotSupportedException` 429
cloning objects 429
`Closeable` 414
closing an I/O stream 1238, 1243, 1244
closing delimiter 460
code corruption 1610
code injection 1609
code migration
bottom-up strategy 1213
top-down strategy 1214
code optimizations 231
Code Ranch (**`www.coderanch.com`**) 1620
code reuse 3, 193, 197
codebase 1612
`Collection` 646
`Collection` interface 786
collections 588, 646, 785, 786
adding elements 865

array operations 800
ArrayList 804
as single entity 646
basic operations 792, 833
bulk operations 792, 841
Cloneable 789
compare with streams 890
data structures 790
duplicate elements 803, 806
elements 646, 785
filtering 798
first element 812, 813, 1490
HashSet 808
implementations 785, 787, 790
interfaces 785
iteration 793, 794
iteration order 797
iterators 795
last element 812, 813, 1490
LinkedHashSet 808
LinkedList 804
List 804
lists 803
map views 841
maximum 864
minimum 864
NavigableSet 812
optional operations 792
ordered 646, 791, 803
parallel streams 899
range-views 803, 812, 813, 1490
retaining elements 793
sequential streams 899
Serializable 789
Set 806
shuffle elements 862
sorted 791
SortedSet 812, 813
sorting 646
subset 812, 813, 1490

text representation 797
thread-safety 804, 842
`TreeSet` 814, 849
unmodifiable view 858
unordered 791
unsupported operations 833
utility methods 785
`Vector` 804
views 658, 812, 813, 841, 1490
collections framework 785
`Collections` utility class 860, 863
`Collector` interface 892, 980
`Collectors` 892, 980
importing 980
collectors 950, 966, 980
averaging 1003
classification mapping 981
classification maps 981
classifier function 981
collecting to a `Collection` 981
collecting to a `ConcurrentMap` 985
collecting to a `List` 982
collecting to a `Map` 983
key mapper 983
map supplier 984
merge function 983
value mapper 983
collecting to a `Set` 982
comparison with terminal operations 1011
composing 981
concurrent 985, 991
counting 1000
downstream 981, 987, 988, 994, 995, 996, 999, 1000
filtering 994
downstream collector 994
predicate 994
finding min/max 1001
finishing 999
downstream collector 999
finisher function 999

flat mapping 996
downstream collector 997
mapper function 997
functional reduction 1000, 1005
grouping 981, 987
classification map 987
classifier function 987
default downstream collector 987
downstream collector 988
map supplier 988
multilevel 989
to a `ConcurrentMap` 991
joining to a string 986
mapping 995
downstream collector 995
mapper function 995
partitioning 981, 991
classification map 992
default downstream collector 992
downstream collector 992
multilevel 993
partitioning predicate 991
reducing 1005
binary operator 1005
identity value 1005
mapper function 1005
summarizing 1004
summary of 1007
summing 1002
colon notation (:) 158
columns 1516
command line 21, 142
command prompt 21
commands
`java` 21
`javac` 20
comments 40
common memory space 1371
common prefix 872, 873
communication 127

compact canonical record constructor 306, 308
compact form styles 1124
compact number formatting 1124
compact number parsing 1125
compact strings 442
`CompactNumberFormat` 1119
`Comparable` interface 434, 449, 763
  `compareTo` 763
comparable objects 763
`Comparator` interface
  `compare` 771
  functional interface 771
comparators 763
  conditional 767, 773, 775
  reverse order 861
comparing objects 428, 746, 771
  see also **equality**
comparing strings 449
comparison 74
compilation unit 330
  deriving permitted direct types 319
compiler
  `-Xlint:unchecked` option 579
compiler optimizations 1457
compiling and running a service 1207
compiling Java source code 20
complement
  ~ 83
completes abruptly
  see **exception propagation** 369
component fields 304
component list 302
composite object 13
composition 16, 197
  see **aggregation**
compound declaration 146
compound messages
  conditional formatting 1150
  formatting 1147
  handling plurals 1151

compound mutually exclusive operations 1484
compound state 1386
compound statement 101
concatenation of strings 450
concatenation operator 67
concise `try` -with-resources 413
concrete class 220
concrete method 228
concrete parameterized types 582
concrete type 575, 582, 588
concurrency 1372, 1544
concurrent activities 1371
concurrent collections 1486, 1487
  blocking 1487
  bounded 1487
  duplicate elements 1487
  happens-before relationship 1487
  non-blocking 1487
  ordering 1488
  unbounded 1487
concurrent collector 985, 991
concurrent maps 985, 991, 1494
  segments 1495
`ConcurrentHashMap` 1495
`ConcurrentLinkedDeque` 1490, 1491
`ConcurrentLinkedQueue` 1490, 1491
`ConcurrentMap` 985, 991, 1495
`ConcurrentModificationException` 660, 797, 798
`ConcurrentNavigableMap` 1496
`ConcurrentSkipListMap` 1495
`ConcurrentSkipListSet` 1490
condition
  `Boolean` 154
  expressions 154
conditional 74
  operators 90
  statements 154
conditional *and* operator 80
  pattern variable 279
conditional comparator 767, 773, 775

conditional expressions 90
associativity 92
nested 92
precedence 91
short-circuit evaluation 91
side effects 91
conditional formatting 1119, 1150
conditional operators 80
conditional *or* operator 80
pattern variable 279
conditions 74
configuration 5
configuration data 1104
congruent 764
connecting punctuation character 30
connection 1523
`Console` 1157
console 1260
formatted prompting 1261
printing formatted values 1261
reading password 1260
`Console` class 1260
`const` 32
constant 548
constant expression 47, 56, 69, 159, 231, 548, 1566
labels 159
constant field values
`case` labels 164
constant string expressions 164
constant values 32, 231
constant variable 56, 231, 548
constants 256
constant-specific class bodies
using anonymous classes 296
constituent objects 13
constituents 16
constraints 1516
constructing array 119
constructor chaining 212, 215, 557
constructor references 727, 732

constructors 7, 109
access modifier 109
accessibility 354
body 109
canonical 304
chaining 212, 215
class name 109
declaration 99
default 110
header 109
implicit default 110
local declarations 109
no-argument 110, 212, 215
non-zero argument 111, 216
overloading 112
superclass constructor 110
constructs 30
high-level 30
loops *see* **iteration statements**
`synchronized` statements 1395, 1396
`Consumer` functional interface 711
composing consumers 712
primitive type specializations 713
contained annotation type 1566
containers 588
see also **collections**
containing annotation type 1566, 1580
contains characters 454
context switching 1372
contextual keywords 31
`non-sealed` 314
`permits` 314
`sealed` 314, 317
`yield` 166
`continue` statement 184
contract 6, 197, 220, 242, 326
control flow
`break` 159, 182
`continue` 184
`do-while` 175

for(;;) 176
for(:) 178
if 154
if-else 155
iteration *see* **iteration statements**
loops *see* **iteration statements**
return 186
statements 101
switch expression 166
switch statement 157
throw 388
transfer statements 181
while 174
yield 166
control transfer 181
conversion categories 47
conversion contexts 47
conversions 263
assignment 47
contexts 46
explicit 43
identity 67
implicit 43
implicit narrowing 67
method invocation 48
narrowing reference 272, 587, 607
numeric promotions 49
parameters 129
reference casting 271
string concatenation 68
to strings 456
truncation 57
type-safe 266
unsafe casts 273
widening reference 198, 272, 586
converting values 433, 434, 436, 437, 439, 456
copy-on-write collections 1505
CopyOnWriteArrayList 1506
CopyOnWriteArraySet 1506, 1507
CopyOption interface 1305

Core Collections Interfaces 788, 1489, 1499
core interfaces 785
Core Map Interfaces 789
core stream interfaces 892
count-down latch 1478
`CountDownLatch` 1478
counter-controlled loops 176
country codes 1101
covariant return 199, 203, 621
`-cp` option
see **class path**
CPU core 1012
CR *see* **carriage return**
crab 178
`CREATE` statement 1518
creating
objects 92
threads 1374
creating a runtime image 1208
creating modular JAR 1193
creating records 305
critical section 1392
CRUD operations 1518
currency 1100, 1120
currency symbol 1130
formatting 1120, 1123
currency formatter 1121, 1123
currency symbol 30, 1130
current directory 1293
`.` 340, 1292
system property 1296
Current Era (CE) 1033
current object 106
currying functions 725
partial application 725
cursor 803, 1538
cyclic barrier 1475
all-or-none model 1478
barrier action 1475
barrier point 1475

`CyclicBarrier` 1475

D

`-d` option 337
daemon 1381, 1406
dangling references 535
Data Definition Language (DDL) 1518
data encryption 1608
data integrity 1610
Data Manipulation Language (DML) 1518
Data Query Language (DQL) 1518
data source 886
data streams 1240, 1242
data structures 646, 790
data types *see* **types**
database capabilities 1547
database connection 1523
database cursor 1538, 1544
Database Management System (DBMS) 1516
database protocol 1524
database transactions 1549
`DataFormat` 1119
`DataInputStream` class 1239
`DataOutputStream` class 1240
`Date` 1028, 1082, 1145
date 763, 1100
    see **temporal objects**
date fields 1028
date units 1044
`DateFormat` 1145
date-time
    see **temporal objects**
date-time formatters
    format styles 1136
    immutability 1132
    ISO-based default 1131, 1133
    ISO-based predefined 1131, 1133
    letter pattern 1139
    parsing 1132

pattern letters 1139
pattern-based 1138
style-based 1131, 1135
thread-safety 1132
DateTimeException 375, 1029, 1037, 1079
DateTimeFormatter 1119, 1131
DateTimeParseException 376, 1055, 1062, 1070, 1135, 1138
daylight saving time (DST) 1077, 1086
time gap 1086
time overlap 1086
time zone rules 1086, 1088
vs. standard time 1086
DayOfWeek enum type 1037
DBMS 1516
DDL 1518
DDL operations 1518
deadlocks 1412
decimal separator 1129
DecimalFormat 1119, 1129
declaration 99
declaration annotations 1575
declaration context 495, 1575
declaration statement 8, 65, 70, 81, 102, 103, 178
declaration-before-reading rule 549
declarations
arrays 92, 118
classes 99, 327
interfaces 327
local 101
main method 141
methods 100
multidimensional arrays 122
packages 327, 330
statements 101
variable arity method 136
declare-before-use rule 574
declared type 205, 266
see **static type**
declared-type parameters 682
declaring *see* **declarations**

decoupling 285
decrement operator 69
deep copying 428
deep reflection 1195
default
    constructor 110
    exception handler 369
    values 103, 543, 557
`default` 248
    label 158, 163, 168
    method 248
default accessibility for class members
    see **package accessibility for class members**
default accessibility for top-level types
    see **package accessibility for top-level types**
default constructor 110
default downstream collectors 987
default file system 1293
default locale 1102, 1103, 1108, 1111, 1113, 1123
default method 248, 677, 678
    conflict resolution 250
    multiple inheritance of implementation 250
    overriding 249
default ordering
    see **natural order**
default package 330
default resource bundle 1106, 1107, 1113, 1114
default values 1566, 1569
defensive copying 361
deferred execution 681, 686
definite assignment analysis 105, 234
definitions
    interfaces 240
    polymorphism 280
`DelayQueue` 1500
`DELETE` statement 1520
delimiters 1124
Denial of Service (DoS) 1606
dependency cycles 1176
deprecated 431

deprecated code
@Deprecated 1584
deprecation warning 1585
ordinarily deprecated 1584
terminally deprecated 1584
deprecation warning 1585, 1587
Deque 823
deque 787, 790
ArrayDeque 825
Deque 823
FIFO queue 823
head 823
LinkedList 825
tail 823
Derby database 1524
connection 1524
derived class 193
deserialization 1615
deserialization of objects 1265
destination directory 337, 1191, 1223
destination object 1157
destination of data 1237
destination stream 25
destructive collection operations 793
diagrams
class 6
object 9
see also **UML**
diamond operator (<>) 572
dictionary 787
dictionary order 449
difference 806
direct sealed superclass 315
direct superclass 213
directory
copy entire directory 1356
see also **directory entry**
directory entry 1292
accessing individual file attributes 1325
atomic move 1317

copy entire directory 1356
copy options 1312
copying 1312
copying using I/O streams 1315
creating 1343
creating regular directories 1346, 1347
creating regular files 1343, 1344
creating streams 1349
creating symbolic links 1346
creating temporary directories 1346, 1348
creating temporary files 1343, 1345
creation time 1328
deleting 1311
denotes same as another entry 1310
determining existence 1309
determining file accessibility 1328
determining kind of entry 1327
determining last modification time 1328
determining owner 1329
determining size 1327
handling POSIX file permissions 1329
hidden 1328
last access time 1328
last modified time 1328
managing file attributes 1325
moving and renaming 1316
reading and writing bytes 1322
reading and writing character data 1319
stream to read text lines 1350
directory hierarchy
breadth-first traversal 1352
cyclic path dependency 1356
depth 1351
depth-first traversal 1352
finding/searching 1358
limit traversal by depth 1355
root 1351
siblings 1351
traversal strategies 1352
walking/traversing 1354

directory-level separator 1193
DirectoryNotEmptyException 1311
disjunct collections 810
dispensing canned advice application 1183
distance of shift 86
distributed 4
divide-and-conquer
    base case 1452
    forking 1452
    joining 1452
    parallel processing 1452
divide-and-conquer algorithm 870
divide-and-conquer paradigm 1451
dividend 62
division
    floating-point 62
    integer 62
division operator
    / 62
divisor 62
DML 1518
documentation 40
documentation comment 40
    tags 40
documenting *see* **documentation**
Domain Name System (DNS) 329, 1175
DosFileAttributes interface 1334, 1337
DosFileAttributeView interface 1338, 1342
dot 328
double 36, 42
double quote 38
DoubleBinaryOperator interface 724
DoubleConsumer functional interface 711
double-ended queues
    deque 823
DoubleFunction interface 714
DoublePredicate functional interface 705
DoubleStream interface 891
DoubleSummaryStatistics 892
DoubleSupplier functional interface 701

`DoubleToIntFunction` interface 715
`DoubleToLongFunction` interface 715
`DoubleUnaryOperator` interface 722
composing unary operators 723
doubly linked list 790
`do-while` statement 175
downcasting 45
downstream collectors 981, 988, 1006
default 987
filtering adapter 994
finishing adapter 999
flat mapping adapter 996
mapping adapter 995
DQL 1518
dropping stream elements 912, 915
duplicate elements 803, 806
duplicating objects 429
duration
accessing units 1071
comparable 1072
compared with period 1076
creating 1069
equality 1072
get methods 1071
immutable 1061, 1068
ISO format 1071
parsing 1070
plus/minus methods 1073
supported `ChronoUnit` s 1070, 1072, 1074
temporal arithmetic 1073
thread-safe 1061, 1068
time-based 1061, 1068
unit conversion 1072
`with` methods 1073
`Duration` class 1068
see **duration**
dynamic 4
dynamic arrays 647
dynamic binding
see **dynamic method lookup**

dynamic method lookup 208, 280, 285
dynamic type 198, 205
dynamically resizable arrays 804

E

eager execution 889
effectively `final` 516, 689
element type 118
element type values for `@Target` 1573
elements 118, 646, 785
elements of collections 785
eligible for garbage collection 536
ellipsis 137
`else` clause matching 157
embedded applications 2
empty set 806
empty statement 101
empty string 443
encapsulation 3, 326, 328, 503, 1167, 1610
    nested types 495
encapsulation of implementation 197
enclosing class 494, 502, 504
enclosing context 495, 502, 507
enclosing instance 495, 500, 526
enclosing object reference 505
encoding of characters 1245
encounter order 893
end of file
    see **end of I/O stream**
end of file (EOF) 373
end of I/O stream 1238, 1244, 1247, 1256, 1257, 1265
ends with characters 454
enhanced `for` loop 174
enterprise applications 2
entries 787, 832
entry point 19, 22, 141, 1193, 1194, 1195, 1216
entry set 833, 1384, 1392
enum class
    see **enum types**

enum constants 289
symbolic names 289
values 289
enum types 165, 258, 289, 334, 637
abstract method declarations 296
accessibility 347
canonical form 290
constant-specific class bodies 296
declaring 290
enum constants 289
enum constructors 292
equality 295
final methods 295
implementing interfaces 300
implicit static methods 294
in `switch` expression 299
inherited methods 295
initializer blocks 293
local 496, 521
members 292
named constants 289
natural order 295
nested 494
ordinal value 165, 295
permitted direct subtypes 318
selector expression 158
static member 496, 497
top-level 493, 496
using 291
enumerated types
see **enum types**
`EOFException` 373
epoch 1028, 1053
epoch-second 1053
equality 75, 428, 746
`equals` method 77, 428, 746
object value 77
objects 77
primitive values 75
reference values 76

`equals` method 77, 428, 746
equivalence relation 748
equivalent collections 810
erasure 615, 618, 621
    of the supertype method 623
    same 622
`Error` 376
escape character 1105
escape sequences 38, 461
essential whitespace 464
evaluation order 52, 80
    arithmetic expressions 59
evaluation short-circuits 80
event listeners
    anonymous classes 523
exact integer arithmetic functions 483
exam 1619
    multiple-choice 1625
    Oracle exam attempt 1621
    program 1622
    questions 1623
    registration 1620
    result 1623
exam objectives
    OCP Java SE 11 1633
    OCP Java SE 17 1627
exam question assumptions
    OCP Java SE 17 1623
`Exception` 372, 374
exception handler 367
    see also **exceptions**
exception handling
    advantages 418
exception type 390
exceptions 367, 378
    `ArrayStoreException` 800
    backtrace 407
    cause 407
    chaining 407
    checked 373

ClassNotFoundException 1266
ConcurrentModificationException 797
customized 377
default handler 369
detail message 372
EOFException 1242
FileNotFoundException 1240
handler 367
IllegalThreadStateException 1382
IOException 1238, 1247
propagation 367
rethrowing 403
situations 372
suppressed 417
throw 388
throwing *see* **throwing exceptions**
thrown by JVM 372
thrown by method 100
thrown programmatically 372
throws 390
types 370
uncaught 369
unchecked 376
UnsupportedOperationException 792
exchanging information 127
execution 1374
execution paths 1371
execution stack
see **JVM stack**
Executor 1428
executor 1427
Executor Framework 1427
executor service 1431
bulk task execution 1443
creating 1433, 1435
executor lifecycle 1431
shutdown 1433, 1435, 1437
submitting tasks 1433, 1435
task cancellation 1448
task execution policy 1430

tasks 1427
termination 1433, 1435, 1437
thread pool 1429
Executors 1428, 1429
ExecutorService 1431
explicit garbage collection 539
explicit module 1210, 1212
explicit referencing 517
exploded module 1185
exploded module directory 1185
exponent 36
exponential functions 482
exports directive 1173, 1175, 1182
accessibility 1177
exports-to directive 1182
expression statements 55, 70, 101, 162, 177, 178, 684
expressions 168
actual parameters 128
boolean 74
conditional 90
deterministic evaluation 50
return 186
statements 101
switch expression 166
throw 388
yield 166
extending 523
inner class 510
interfaces 246
Thread 1379
extends 583, 591, 593
bounded type parameters 593
constraint 593
extends clause 13, 193
extensibility 1611
extensions
.class 21
.java 21, 22
external iteration 891
extracting substrings 455, 468

F

fairness 1412
fairness policy 1465, 1474
  acquisition order 1465
fallback 1114
fall-through 158, 159, 167
`false` literal 32, 37
fetch size 1540
FF *see* **form feed**
field declarations 99
field hiding 205, 233
field initialization 557
fields 6
  `final` 231
  hiding from superclass 205
  shadowing by local variables 115
FIFO queue 823
file 763
  see also **directory entry**
file attribute views 1333, 1337, 1338
  view name 1332
file attributes 1325
  accessing by attribute name 1331
  accessing by bulk operations 1332
  accessing individually 1325
  file attribute name format 1332
  file attribute views 1333
  file attributes interfaces 1334
file attributes interfaces 1332, 1334
`File` class 1294, 1296
file extension 1292
  `.properties` 1107
file name 327
file path 337
  separator character 337
file streams 1240
file system 1237, 1291
  accessing 1614
  default 1293

file system resources 1349
file visit option 1353
FileAlreadyExistsException 1313, 1343
FileAttribute interface 1343
FileAttributeView interface 1338
FileInputStream class 1239
FileNotFoundException 373
FileOutputStream class 1240
FileOwnerAttributeView interface 1338
FileReader class 1247
files
    binary 1242
    low-level I/O streams 1240
    not found 1240
Files class 1291
    method characteristics 1308
    streams 905
FileSystem class 1293
FileSystems class 1293
FileTime class 1328
FileVisitOption enum type 1353
FileWriter class 1248
filtering 690
filtering adapter 994
filtering streams 912
    using a Predicate 914
FilterInputStream class 1239
FilterOutputStream class 1240
filters 912
    see also **I/O filter streams**
final
    effectively 694
    variables 694
final fields 231, 232
    hiding 233
final instance fields
    initialization 233
final keyword 227
final local variables 231, 233
final members 227

`final` methods 228
binding of method call 229
`final` parameters 136, 234
`final static` fields
initialization 233
manifest constant 233
`final` variables 231
blank 231
constant variables 231
definite assignment analysis 234
definitely assigned 234
definitely unassigned 235
fields 231
local variables 231
object state 231
`finalize` method 429
`finally` clause 379, 384
finishing adapter 999
finite streams 894
extracting lines from a `String` 904
from a `BufferedReader` 904
from a `CharSequence` 903
from arrays 900
from collections 899
from factory methods in the `Files` class 905
from specified values 895
from the `Random` class 902
first element 812, 813, 1490
first map entry 848
fixed arity method 136
fixed arity method call 140
Fixed Thread Pool 1430
fixed-size list view 661
flat mapping adapter 996
`float` 36, 42
floating-point 41
`double` 42
`float` 42
literals 36
floating-point arithmetic 60

floating-point data types 36
floating-point division 62
floating-point remainder 63
floor entry 848
floor key 848
flow control *see* **control flow**
flushing I/O streams 1238
`for(;;)` statement 176
    backward 177
    collections 796
    forward 177
    iterating over array 121
    iterator 796
`for(:)` statement 178, 797
    iterating over array 121
for-each loop 174
Fork/Join Framework 1012, 1451
forking 1452
fork-join pool 1453
`ForkJoinPool` 1452
`ForkJoinTask` 1452
form feed 38, 39
formal parameters 100, 109, 128, 267, 356
    modifier 100
    name 100
    type 100
formal type parameter 569
`Format` class 1119
format elements 1144
    format style 1144
    format type 1144
format specifications 25, 459
format specifiers 25, 1251
format style 1131, 1136, 1144
format type 1144
`FormatStyle` enum type 1131, 1135
formatted output 24
    format specifiers 25
formatted prompting 1261
formatted string 459

formatted values
Console 1261
PrintStream 1251
PrintWriter 1251
Formatter 1157
formatter
destination object 1157
formatting 39, 1119
formatting date and time values 1131
formatting messages 1143
conditional formatting 1150
formatting numbers/currency values 1120
forward reference 543, 544, 547, 553, 557
fractional signed numbers 41
frame stack
see **JVM stack**
framework for collections 785
full permissions 1347
fully 1195
fully qualified class name 339, 1193, 1194, 1195
fully qualified name 316, 347, 499
fully qualified package name 328, 330, 331
fully qualified type name 328, 333
function 686
function arrow -> 681
Function interface 714
composing functions 717
function type 685
functional interface
@FunctionalInterface 677, 1583
abstract method 677
built-in functional interfaces 697
function type 685
functional method 677
primitive values 698
target type 684
see also **lambda expressions**
functional method 677
functional programming 5
functional reduction 957, 958, 974, 986, 1000

functional requirements (FRs) 1604
`FunctionalInterface` annotation 677
functionality 677
functional-style programming 677
functional-style programming paradigm 3
functions
    currying functions 725

G

garbage collection 538, 539
    automatic 535
    facilitate 539
general abstractions 196
general contract
    `compareTo()` 764
    `equals()` 748
    `hashCode()` 758
general loops 176
generalization 13
generalized classes 196
generic class 569, 570
generic instance method 596
generic interface 574
generic method 595, 612, 659
    calling 596
generic method call
    inference 596
generic static method 596
generic subtype method 623
generic type 569, 647
    extending 575
    implementing 600
generics 147
    arrays 629
    casting 627
    class literals 638
    collections 580
    constructors 595
    enums 637

erasure 615, 622
exception handling 634
inherited methods 622
`instanceof` operator 626
JVM 567
method signature 617
methods 595
nested classes 635
non-reifiable types 626
overloading 617, 618
overriding 617, 619
reifiable types 625, 626
subsignature 618
type erasure 578, 615
variable arity parameter 632
wildcards 581
GESS 592
get methods 302, 304, 360, 592, 977, 1036, 1103, 1535, 1538, 1540, 1541
get operation 588
getter methods *see* **get methods**
GMT (Greenwich Mean Time) 1077
`goto` 32
grammar rules 30
grant 1612
graph
nodes 1170
unidirectional edges 1170
grouping 328
grouping separator 1129
groups of objects 785
GUI applications 1382

H

half-open interval 867, 869, 1150
handle-or-declare rule 390
handles *see* **references**
happens-before relationship 1418, 1458
transitivity 1418
hard link

see **symbolic link**
has-a relationship 196
hash code 108, 428, 755, 756
hash function 755
hash sign 1105
hash table 756, 790, 791
see **map**
hash value 428
see **hash code**
hashing 755
collision 755
hash code 755
HashSet 808
Haskell Curry 726
head map view 847, 848
head set view 812, 813, 1490
heap 535, 1374
heap pollution 1589
heap pollution warning 632, 652, 661, 807, 835, 865
helper methods 254
heuristics 758
hidden members
accessing 507, 508
hiding internals 326
hierarchical file system 1237
higher entry 848
higher key 848
high-level I/O streams 1237
high-performance 5
holdability 1544
horizontal tab 38
hotspots 5
HT *see* **horizontal tab**
human time 1053

I

I/O buffering 1242
I/O filter streams 1239, 1240, 1242

I/O package 1237
I/O streams 1237
buffering 1242
chaining 1242
closing 1238, 1243, 1244
end of stream 1244
files 1240
filter streams 1242
flushing 1238
high-level 1237
input filter streams 1239
low-level 1237
of binary values 1240, 1242
of characters 1245
of objects 1240
output filter streams 1240
skipping bytes 1242
IANA Time Zone Database 1077
IDE (integrated development environment) 1620
idempotent method 410, 415
identifiers 30
variable 102
identity container 967
identity conversion 46, 67
identity of object 8
`identity()` function 984
IEEE 754-1985 42
`if` block 154
`if-else` statement 155
scope of pattern variable 278
illegal thread state 1382
`IllegalArgumentException` 374, 812, 1139
`IllegalStateException` 375, 796
immediately enclosing instance 494
immutability 1457, 1611
immutable 304, 798, 1028, 1079
immutable classes
in Java SE platform documentation 358
immutable object pattern
see **immutable objects**

immutable objects 431, 442
advantages 362
caching 360
consistency of state 360
defensive copying 361
guidelines 360
state 358
static factory method 361
thread-safety 358
immutable primitive values 358
immutable strings 442
implementation inheritance 247
implementations 6, 195, 326
collections 785
inheritance hierarchy 228
implementations of collections 787
implementing
`Comparable` 763
`Comparator` 771
equals 746
hashCode 755
interfaces 242
`implements` clause 242
implicit canonical record constructor 304
implicit default constructor 110
implicit dependency 1171, 1176, 1220
implicit inheritance 193
implicit narrowing conversions 67
implied dependency 1179
`import` 347
declaration 331
see also **static import**
single-type-import declaration 331
statement 327
type-import-on-demand declaration 332
importing
enum constants 334
nested types 500
reference types 331
static members 333

in lockstep 799
IN parameters 1534
incidental whitespace 464
increment operator 69
indenting lines 458
index 118
index expression 121
`IndexOutOfBoundsException` 402, 446, 447, 456, 467, 469, 470, 653, 654, 657
individual array elements 121
individual module compilation 1190
inequality 75
    see also **equality**
inferred-type lambda parameters 147
inferred-type parameters 682
infinite loop 178
infinite streams 894
    building with generator functions 896
    from the `Random` class 902
infinity 60, 434
    negative 60
    positive 60
information hiding 326
inheritance 13, 196, 240
    hierarchy 195, 509
    serialization 1274
    subtype—supertype relationship 197
inheritance of implementation 240
inheritance of type 240
initial capacity 649
initial state of object 557
initialization
    arrays 120, 125
    code 120
    default values 103
    `for` statement 176
    objects 8
    references 102
    variables 102
initialization expression 102, 143
initializer 542

declaration-before-reading rule 544
static 543, 544, 555, 556, 559
initializer blocks
instance 553
static 547
initializer expression 103, 231, 232, 542
static 545
initializers
declaration order 547
exception handling 551
forward references 547
non-static block 99
non-static field 99
static block 99
static field 99
using class name 549
using simple name 547
initializing *see* **initialization**
inlining 229
inner classes 493, 494
anonymous 494, 496, 523
local 494, 496, 514
non-static member 494, 496
qualified superclass constructor invocation 511
synchronization 1396
INOUT parameters 1536
input 1237
input stream 1237
input validation 1609
`InputStream` abstract class 1238
`InputStreamReader` class 1247
`INSERT` statement 1519
insertion order 646, 791, 809, 844
insertion point 863, 869
instance
members 13
methods 13, 101, 106
variable initialization 103
variables 13, 106
see also **object**

instance initializer blocks 553
local variable type inference 554
instance methods 9
instance variables 8, 557
`instanceof` operators 92, 271, 276, 626
`instanceof` pattern match operator 276
pattern variable 277
type pattern 277
`instanceof` type comparison operator 272, 283
instant
accessing fields 1056
comparing 1057
convert to temporal objects 1060
creating 1053
creating from date-time 1055
current instant 1054
epoch 1053
epoch-second 1053
immutable 1053
ISO format 1054
max value 1053
min value 1053
nano-of-second 1053
parsing 1055
plus/minus methods 1058
representation 1053
supported `ChronoField` s 1058
supported `ChronoUnit` s 1060
temporal arithmetic 1058
text representation 1055
thread-safe 1053
timestamps 1053
`with` methods 1058
zero UTC offset 1053
`Instant` class
see **instant**
instant timeline 1078
instantiation 8
anonymous classes 526
local classes 519

non-static member classes 504
parameterized type 571
static member classes 497
`int` 32, 42
`IntBinaryOperator` interface 724
`IntConsumer` functional interface 711
integer arithmetic 59
integer bitwise operators 82
integer constant expressions 48
integer data types 32
integer division 62
integer remainder operation 62
integers 41
and operator 83
`byte` 42
complement 83
data types 41
`int` 42
literals 32
`long` 42
or operator 83
representation 54
`short` 42
types 41
xor 83
integral types 41, 42, 44
interface constant antipattern 334
interface-based 789
interfaces
abstract 241
abstract methods 242
accessibility 347
body 240
collection core 785
constants 256
contract 242
declarations 327
default methods 248
evolution 252
extending 246

functional interfaces 677
header 240
implementing 242, 300, 525
inheriting from multiple interfaces 623
initialization 559
local 496, 521
markers 241
member declarations 240, 241
multiple inheritance of implementation 250
nested 493, 494
overriding default methods 249
partial implementation 244
private methods 254
public methods from `Object` class 247
realization 246
references 248
static member 496, 497
static methods 253
subinterfaces 246
superinterfaces 246
top-level 493, 496
UML 246
UML stereotype 246
variables 256
with single abstract method 680
intermediate stream operations 887, 907
examining elements 922
filtering streams 914
flattening streams 926
input stream 887
lazy execution 889
mapping streams 923
non-interfering behavioral parameters 912
order of 910
output stream 887
replacing each element with multiple elements 929
selecting distinct elements 917
setting execution mode 935
setting stream as unordered 934
short-circuit execution 910, 919, 920

skipping elements 917
sorting streams 931
stateful operations 910
stateless behavioral parameters 912
stateless operations 910
truncating a stream 919
internal iteration 891
internal packages 1175
internationalization 1100
interned strings 442, 444
interned values 435
interpackage accessibility 326
interrupt handling 1397
interrupt status 1397
InterruptedException 1397
spurious wakeup 1403
InterruptedException 1397
interruptible locking 1466, 1471
intersection 806
IntFunction interface 714
IntPredicate functional interface 705
intrinsic lock
see **object lock**
intrinsic locking 1392, 1464
IntStream interface 891
IntSummaryStatistics 892
IntSupplier functional interface 701
IntToDoubleFunction interface 715
IntToLongFunction interface 714
IntUnaryOperator interface 722
composing unary operators 723
invocation
parameterized type 571
invocation stack
see **JVM stack**
invoker 186
invoking garbage collection 539
IOException 373
is-a relationship 196, 197
ISO 8859-1 encoding scheme 441

ISO Latin-1 37, 42
ISO standard 1131, 1133
Iterable interface 452, 659, 794, 797
iteration 176, 794
iteration order 797
iteration statements 174
next iteration 184
termination 174, 183
iterators 647, 659, 795, 799
fail-fast 796, 1489
ListIterator 803
ordered collection 798
snapshot-style 1488
sorted collection 798
unordered collection 798
weakly consistent 1488

J

Jakarta EE (Enterprise Edition) 2
JAR 1167, 1193
jar tool 1193
java 21
Java Application Launcher 21
see **JDK tool** java
Java Archive
see **JAR**
Java Archive Tool
see **JDK tool** jar
Java bytecode 21
Java Card 2
Java Class Dependency Analyzer Tool
see **JDK tool** jdeps
Java Collections Framework 646
Java compilation unit 1172
Java Database Connectivity
see **JDBC**
Java Development Kit (JDK) 2
Java ecosystem 2

Java EE (Enterprise Edition) 2
Java Language Compiler 20
see **JDK tool** javac
Java Linker
see **JDK tool** jlink
Java ME (Micro Edition) 2
Java Platform Module System 1167
Java Platforms 2
Java Runtime Environment (JRE) 3
Java SE (Standard Edition) 2
Java Shell Tool (jshell) 23
Java Virtual Machine (JVM) 2
java.base module 1171, 1218
java.io package 1237, 1294
java.lang.annotation package 1561
java.lang.Record class 306
java.lang.reflect package 1591
java.math package 487
java.net package 1294
java.nio package 1291
java.nio.file package 1291
java.nio.file.attribute package 1328
java.se module 1172
java.sql package 1521
java.text package 1119
java.time package 1028, 1119
java.time.format package 1119
java.util package 646
java.util.concurrent package 1428
java.util.function package 697
javac 20
Javadoc comment 40
@param tag 100, 112
@return tag 186
@throws tag 392
javadoc tool 40
JDBC 1521
JDBC API 1521
JDBC drivers 1521
legacy drivers 1526

JDBC interfaces 1523
JDBC resources 1522
JDBC URL 1523
JDK 1620
licencing 3
JDK tools
jar 1193, 1215, 1216, 1217, 1225
java 21, 1193, 1216, 1217, 1224
javac 20, 1190, 1223
javadoc 40
jdeps 1171, 1218, 1226
jlink 1208, 1226
jmod 1170
jshell 23
summary of operations 1222
JMOD archive 1170
JMOD files 1217
join
starvation 1416
joining 1452
joining threads 1387
JPMS 1167
just-in-time (JIT) 4
JVM 536, 539
heap 1374
JVM stack 1374
method area 1374
program counter (PC) 1374
runtime constant pool 1374
runtime data areas 1373
JVM stack 367, 536, 1374

K

key 868
key insertion order 843
key objects 787
key sets 833, 849
key sorting 847
key-based lookup 1108

keys 832
keystore 1612
key—value entries 756
key—value pairs 1106
keywords 31
abstract 221, 242, 677
boolean 43
break statement 182
byte 42
case 158, 162, 166
catch 379
char 42
class 99, 240
contextual words 31
continue 184
default 158, 163, 168, 248
do 175
double 42
else 155
extends 193, 583, 591, 593
final 136, 227, 234, 314, 360, 402, 412, 1460
finally 384
float 42
for 176, 178
if 154
implements 242
import 331, 347
instanceof 92, 272, 276, 283, 626
int 42
interface 240
long 42
new *see* new **operator**
null 49, 77, 271
package 330, 347
private 353
protected 352
public 352
reserved words 31
return 186
short 42

static 19, 113, 253, 333, 497, 547
super 110, 203, 208, 213, 250, 509, 584
switch 157, 166
synchronized 1392, 1460
this 106, 506
throw 388
throws 390
try 379
unused words 31
void 19, 431
volatile 1458
while 174, 175
keywords (contextual)
exports 31, 1173, 1181
module 31, 1181
non-sealed 31
open 31, 1181
opens 31, 1181
permits 31
provides 31, 1181
record 31
requires 31, 1174, 1179, 1181
sealed 31
to 31, 1181
transitive 31, 1181
uses 31, 1181
var 31
with 31, 1181
yield 31
keywords (reserved)
const 32
goto 32

L

labeled break statement 183
labels 181, 183
break 183
case constants 158, 162
constant expressions 159

default 158, 163, 168
labeled statement 181
switch statement 158, 162
lambda body 681, 683
lambda expressions 681, 693
access class members 687
block scope 686, 688
compared with anonymous classes 694
contexts providing target types 735
declared-type parameters 682
deferred execution 681, 686
expression 683
expression statement 684
function 686
function arrow -> 681
implicitly typed 682
inferred-type parameters 682
lambda body 681, 683
lambda parameters 682
non- void return 683
parameter list 681
shadowing members 687
single expression 683
statement block 684
target type 684
target typing 686
type checking 684
using this and super 687
variable capture 689
var-type inferred parameters 682
void return 683
lambda parameters 682
language codes 1101
language tag 1122
last element 812, 813, 1490
last map entry 848
latch
see **count-down latch**
late binding
see **dynamic method lookup**

LATIN-1 encoding scheme 441, 442
lazy execution 889, 909
least significant bit 34
left associativity 51
left shift 86
legacy class 568
legacy code 567
legacy supertypes 621
legal assignments 266
length method 447, 904
letter pattern 1139
lexical nesting 498
lexical scope
see **block scope**
lexical tokens 30
lexicographic array comparison 871
lexicographic comparison 871
lexicographical ordering 449
LF *see* **linefeed**
lifetime 537
see **scope** 106
limits 1150
line continuation 38
line separator 25, 1250
line terminator 39, 447
linear implementation inheritance 195
linear inheritance 239
line-continuation character 1190
linefeed 38
lines of text 1256
LinkageError 376
linked data structures 568
linked list 790, 804
LinkedBlockingDeque 1500
LinkedBlockingQueue 1500
LinkedHashSet 808
LinkedList 649, 804, 817, 825
LinkedTransferQueue 1500
LinkOption enum type 1305
List 647, 803

list views 658, 661, 662
listing observable modules 1216
`ListIterator` 803
lists 786, 803, 804
  `ArrayList` 804
  binary search 863
  changing contents 865
  copying 865
  `LinkedList` 804
  `List` 803
  `ListIterator` 803
  positional 787
  replacing elements 864, 865
  reverse order 861
  sorted 863
  sorting 860, 861
  sublist view 658
  sublists 864
  unmodifiable 651, 803
  unmodifiable views 858
  `Vector` 804
  views 661
  see also `ArrayList`
literals 32
  boolean 37
  character 37
  default type 32, 36
  double 36
  escape sequences 38
  `false` 37
  float 36
  floating-point 36
  integer 32
  `null` 32
  predefined 31
  prefix 33
  quoting 37
  scientific notation 36
  string 39
  suffix 32, 36

true 37
live threads 1387
livelock 1414
liveness 1412
local 102, 104, 521
block 494
chaining of constructors 212, 557
classes 496, 514
enum types 494, 496
interfaces 494, 496
record classes 494, 496
variables 106, 356
local classes 494, 496, 514
accessing local declarations 515
accessing members in enclosing context 517
declaration 514
hiding members from enclosing context 517
in non-static context 514
in static context 514
instantiation 519
scope 514
shadowing local variables 516
using qualified this 517
local date
conversion to Date 1092, 1145
local date-time
conversion to Date 1092, 1145
converting to instant 1055
local declarations 101
local enum types 521
local interfaces 521
local record class 521
local scope 171
local time 1077
conversion to Date 1092, 1145
local timeline 1078
local variable type inference 143, 412
arrays 144
instance initializer blocks 554
parameterized types 573

static initializer blocks 547, 549
local variables 109, 143
block scope 686
LocalDate
see **temporal objects**
LocalDateTime class
see **temporal objects**
Locale 1100
locale 450, 1100
country codes 1101
country name 1102
default 1102
language codes 1101
language name 1102
predefined 1101
locale qualifiers 1102
locale-sensitive 1119
locale-sensitive operations 1100
localization 1100
localizing an application 1109
localizing information 326
LocalTime class
see **temporal objects**
locations
see **class path**
Lock 1465
lock 1384
acquire 1384, 1392
class 1395
entry set 1384, 1392
release 1384
lock acquisition
starvation 1416
logarithm 482
logical 279
logical AND
& 78
logical complement
! 78

logical complement (!) operator
pattern variable 279
logical exclusive OR
^ 78
logical inclusive OR
| 78
logical XOR
^ 78
long 32, 42
suffix 32
LongBinaryOperator interface 724
LongConsumer functional interface 711
LongFunction interface 714
LongPredicate functional interface 705
LongStream interface 891
LongSummaryStatistics 892
LongSupplier functional interface 701
LongToDoubleFunction interface 715
LongToIntFunction interface 715
LongUnaryOperator interface 722
composing unary operators 723
loop body 174, 176
loop condition 174, 176
loop variable 176
loops *see* **iteration statements**
loss of precision 44
lower bound 583
lower bounded wildcard
? super T 584
lower bounded wildcard references 591
lower entry 848
lower key 848
low-level I/O streams 1237
LTS (long-term support) release 3

M

magnitude 44
main memory 1457
main method 21, 141

arguments 142
modifiers 141
main thread 1377, 1382
`Main-Class` attribute 1216
`MANIFEST.MF` file 1216
many-to-one relation 832
`Map` 833
map
key-value entries 756
map views 841
`Map.Entry` 835, 848
mapping adapter 995
mappings 787
see **entries** 787
map-reduce operation 1005
map-reduce transformation 960
maps 787, 832
access order 843
entries 787
iteration 841
key insertion order 843
key-based operations 833, 837
`Map` 833, 842
map views 841
mappings 787
multimap 599
`NavigableMap` 847
ordering mode 843
replacing values 841
`SortedMap` 847
submap view 848
text representation 843
`TreeMap` 849
unmodifiable 834
unmodifiable views 859
views 841
marker annotation 1567, 1569
marker annotation type 1564
marker interfaces 241, 1266
marker parameter 1530, 1534

mask
see **bit mask**
`Math` class 108, 480
math constants 480
mathematical set 806
`MAX_VALUE` constant 436
maximum value 481
member declarations 99, 309
member type declarations 99
non-static member class 494
static member class 494
static member enum type 494
static member interface 494
static record class 494
members 6, 354
access 106
accessibility 349
default values 103
inheritance 193
modified 193
of objects 9
scope 354
shorthand 107
`static` 10
terminology 13
variables *see* **fields**
memory consistency errors 1418
memory management 535
memory organization 535
`MessageFormat` 1119
meta-annotation 1563, 1571
`@Documented` 1578
`@Inherited` 1578
`@Repeatable` 1579
`@Retention` 1571
`@Target` 1573
metadata 241, 1561
`META-INF` directory 1215, 1216
method area 1374
method call 9, 100, 127

chaining 469, 471
fixed arity 140
variable arity 140
method chaining 908, 1040, 1046, 1064, 1073
method declaration 99
variable arity parameter 632
method header 226
method invocation conversions 48, 267
method modifiers 100
method naming conventions 1029
method overloading 108, 204, 227
method overriding 204, 226, 228, 558
covariant return 203
method references 726, 727
bounded instance method references 729
static method references 728
unbounded instance method references 731
method signature 101, 198
method type 685
methods 6
`@Override` 200
`abstract` 226, 242, 677
abstract method declarations 242
advisory 1389, 1391
ambiguous call 204, 268
automatic variables *see* **local variables**
behavior 677
binding of method call 207
blocks 101
body 100
bridge 617
call chaining 128
call *see* **method call**
calling variable arity method 138
chained 451
`clone` 428
concrete 228
declaration 100, 128
`default` 248
dynamic lookup 285

`equals` 77, 428, 746
exceptions 100
`final` 228
`finalize` 429
fixed arity 136
functional 677
generic 595
`getClass` 428
header 100
hiding 228
hiding static methods 205
implementation 226
invocation *see* **method call**
local declarations 101
local variables
locale-sensitive 1103
`main` *see* **main method**
method invocation conversions 267
method type 685
modifiers 100
most specific 268, 657
mutually exclusive 1395
name 127
objects 106
overloaded resolution 267
overloading *see* **method overloading**
overriding *see* **method overriding**
overriding vs. overloading 204
parameters 100
`return` 186
return value 100
signature 101, 108, 204, 618
`static` 115, 253
subsignature 619
`synchronized` 1392
termination 186
`throws` clause 390
`toString` 428
variable arity 136

migration to modules 1213
`MIN_VALUE` constant 436
minimizing overhead 538
minimum value 480
`MissingResourceException` 376, 1108
mixing raw types and generic types 577
mobile applications 2
modifiers
  `abstract` 242
  access 347
  `default` 248
  `final` 227
  `static` 253
modular JAR 1193, 1210
  creating 1193
  describe a JAR 1217
  extract contents 1216
  list contents 1215
  listing observable modules 1216
  `Main-Class` attribute 1216
  `MANIFEST.MF` file 1216
  `META-INF` directory 1215, 1216
  running from 1195
modular JDK 3, 1168
  goals 1169
  `java.base` module 1171, 1218
  `java.se` module 1172
modular JDK installation 1169
module application directory structure 1184
module declaration 1172
  `module` as contextual keyword 1173
module dependency 1174
module descriptor 1173, 1191
module directives 1173
  `exports` directive 1173, 1175, 1182
  `exports-to` directive 1182
  `opens` directive 1182, 1195, 1196
  `opens-to` directive 1182, 1199
  overview of directives 1181
  `provides-with` directive 1182, 1204

qualified export 1182
`requires` directive 1173, 1174, 1181
`requires static` directive 1181
`requires transitive` directive 1179, 1181
unqualified export 1173, 1182
`uses` directive 1182, 1206
module graph 1170, 1176, 1184, 1221
acyclic 1176
dependency cycles 1176
implicit dependency 1171, 1176
implied dependency 1179
module dependency 1174
sink nodes 1176
module name 1175
module path 1210
module resolution 1176, 1190
module root directory 1185
`module-info.java` file 1185
modules 1172
accessibility 1177
accessibility rules 1174, 1178
aggregate module 1172
automatic 1210, 1211
benefits 1167
comparison 1212
creating modular JAR 1193
dependencies 1170
explicit 1210, 1212
exploded module 1185
exploded module directory 1185
exploring modules 1215
implicit dependency 1220
individual module compilation 1190
internal packages 1175
migration to 1213
modular JAR 1193
module application directory structure 1184
module declaration 1172
module descriptor 1173
module graph 1170

module name 1175
module resolution 1190
module root directory 1185
`module-info.java` file 1185
multi-module compilation 1191
named 1212
non-standard modules 1170
`open` modifier 1182
platform modules 1170
principle exported package 1175
public API 1167, 1174, 1178
readability 1177
readable 1174
reliable configuration 1167
running from a modular JAR 1195
services 1200
standard modules 1170
stronger encapsulation 1167
system modules 1216
unnamed 1210
viewing dependencies 1218
monitor 1384
entry set 1384
lock 1384
monitor lock
see **object lock**
`Month` enum type 1032
most specific method 268, 657
multi-`catch` clause 399
`final` parameter 402
multicore architectures 5
multidimensional arrays 122, 124
multi-element annotation type 1565
multilevel classification 989
multilevel grouping 989
multilevel partitioning 993
multimap 599
multi-module compilation 1191
multi-paradigm programming 3
multiple assignments 55

multiple bounds 594
multiple `catch` clauses 378, 399
multiple inheritance of implementation 195, 240, 250
multiple inheritance of type 240
multiple-line comment 40
multiplication operator
`*` 61
multiplicative operators 61
multisets 806
multitasking 1371
multithreaded 5
multithreaded programming 1372
mutability 663
mutable character sequences 466
mutable containers 949, 966
mutable objects 361
mutable reduction 973, 974
mutable containers 966
see also **collectors**
mutator method *see* **set methods**
mutex
see **lock**
mutual exclusion 1459
mutually comparable 860, 863, 869
mutually exclusive
actions 156
locks 1384
MVC design pattern 1183

N

name 30
name clash 622, 623, 624
name elements 1292
name separator in path 1292, 1295
named constants 233
named modules 316, 1212
named package 316
namespaces 109, 1175
naming conventions of paths 1292

NaN (Not-a-Number) 61, 434, 1610
nano-of-second 1053
narrower range 44
narrowing conversions
primitive 44
reference 45
narrowing reference conversions 198, 272, 587, 607
natural logarithms 480, 482
natural order 763, 860, 867
natural ordering 867
navigable map 787
first entry 848
last entry 848
range-views 848
views 848
navigable set 786, 812
ceiling 814, 1491
floor 814, 1491
higher 814, 1491
iterator 814, 1491
lower 814, 1491
NavigableMap 847
NavigableSet 812
navigation methods 847
closest match 813
nbsp
see **non-breaking space**
negative zero 61
nested classes 494
anonymous 494, 496, 523
class file name 499
fully qualified name 499
generic 635
local 494, 496, 514
non-static member 494, 496, 503
qualified name 498
static member 493, 496, 497
synchronization 1396
nested enum types
local 496

static member 493, 496
nested interfaces 493
static member 496
nested lists 650
nested loops 126
nested parameterization 577
nested parameterized types 610
nested record classes 494
local 496
static member 496
nested type declaration 493
nested types 493
accessibility 495
enclosing context 495
enclosing instance 495
importing 500
nested wildcards 586
`new` operator 8, 92, 109, 119, 509, 523, 557
qualified class instance creation expression 504
`NEW` state 1386, 1390
newline *see* **linefeed**
NIO.2 1291
NL *see* **newline**
no-argument constructor 110, 212, 215
`NoClassDefFoundError` 376
nodes 568, 1170
non-access method modifier 99, 100
non-atomic operation 1459
non-blocking collection 1487
non-blocking lock-free synchronization 1461
non-breaking space (nbsp) 1121, 1124
non-functional requirements (NFRs) 1604
non-interfering behavioral parameters 912
non-modular JARS
see **plain JARs**
non-observable substate 1386
non-reifiable types 625, 633
non-runnable states 1387
non-standard modules 1170
non-static code 100

see **non-static context** 99

non-static context 99, 100

non-static field 13

non-static field initializers 99

non-static initializer block 99

non-static member classes 496

access hidden members 507

access in enclosing context 506

declaration 503

fully qualified name 499

hiding in enclosing context 509

inheritance 509

instantiation 504

qualified class instance creation expression 504, 505

qualified name 498

qualified superclass constructor invocation 511

qualified `this` 506

non-static nested classes

see **inner classes**

non-static type 570

non-varargs call

see **fixed arity call**

non- `void` return 683

non-zero argument constructor 111, 216

normal annotation 1567

array element initializer 1568

element-value pair 1567

single-element array-valued element-value pair 1568

normal canonical record constructor 306, 307

normal execution 368

normal non-canonical record constructor 307, 309

normal threads 1381

`NoSuchElementException` 795, 813, 817

`NoSuchFileException` 1311

Not-a-Number

see **NAN**

notifying threads 429, 1400

`NotSerializableException` 373

`null` literal 32

casting 49
equality comparison 77
null reference 32
casting 271
nulling references 538
NullPointerException 375
null -safe 745
Number class 436
number format pattern symbols 1129
number formatter 1121, 1123
accounting currency 1122
compact form 1124
currency 1120, 1121
numbers 1120
percentage 1120, 1121
number formatting
accounting currency 1122
compact form 1124
compact form styles 1124
pattern symbols 1129
static factory methods 1120
string pattern 1129
number systems
base 33
decimal 33
hexadecimal 33
octal 33
radix 33
NumberFormat 1119, 1120
NumberFormatException 375, 399, 433
numbers 1100, 1120
formatting 1120
numeric optional classes 946
numeric promotions 49
assignment 56
binary 50
unary 49
numeric streams 887, 894
averaging 976
from a range 900

from arrays 900
from the `Random` class 902
functional reduction 974
summary statistics 974, 976
summation 975
numeric types 41
numeric wrapper classes 436
numerical literals
using underscore 36

O

`ObjDoubleConsumer` functional interface 711
`Object` class 195, 427
object creation 1274
object graph 1270
object hierarchy 197
object lock 1384, 1392, 1395, 1401, 1402, 1403, 1412
Object lock rule 1418
object reference equality 747
object references 8, 102
object serialization
see **serialization**
object state 8, 109, 132, 557
immutable 358, 1457
object streams 887, 894
object type *see* **dynamic type**
object value equality 747
`ObjectInputStream` class 1239
object-level lock
see **object lock**
object-oriented programming 6
object-oriented programming paradigm 3
`ObjectOutputStream` class 1240, 1265
objects 8
aggregate 16
alive 536
arrays 118
behaviors 6
callee 127

caller 127
Class class 428
cleaning up 538
cloning 429
communication 127
comparing 428, 746, 771
compound 536
constituent 16, 536
constructing 557
contract 326
creation 1274
decoupling 285
deserialization 1265
eligible 538
equality 77, 428, 746
exchanging information 127
garbage collection 535
I/O streams 1240
identity 8
immutable 431
implementation 326
initial state 557
initialization 8, 109
initializer block 553
internals 326
lifetime 537
members 9
methods 106
Object class 427
persistence 1265
properties 6
reachable 535, 536
reading 1265
serialization 1265
services 326
state 231
state *see* **object state**
value equality 77
wait set for threads 1400
writing 1265

`Objects` class
comparing objects 745
computing hash code 745
`null` -safe methods 745
`ObjIntConsumer` functional interface 711
`ObjLongConsumer` functional interface 711
observable modules 1217
on/off latch 1479
one's complement 34
one-arity 697
one-dimensional arrays 118
one-shot tasks 1444
one-to-many mapping 926, 929
one-to-one mapping 923
one-to-one transformation 929
online proctored exam 1622
on-the-fly classes 494
`open` modifier 1182, 1200
open module 1200
open package 1196
open range-view operations 803
opening delimiter 460
`OpenOption` interface 1305
`opens` directive 1182, 1195, 1196
`opens-to` directive 1182, 1199
operands 48
evaluation order 52
operations 6
put 588
read 588
write 588
operators 50
`-` 58, 63
`--` 69
`^` 78, 82
`^=` 79, 85
`!` 78
`!=` 75, 76
`? :` 90
`.` 9, 328

[] 92, 121
* 58, 61
*= 66
/ 58, 61, 62
/= 66
& 78, 82
&& 80
&= 79, 85
% 58, 61, 62
%= 66
+ 58, 63, 67
++ 69
+= 66
< 74
<< 86, 87
<<= 90
<= 74
-= 66
= 54
== 75, 76
-> 92
> 74
>= 74
>> 86, 88
>>= 90
>>> 86, 89
>>>= 90
| 78, 82
|= 79, 85
|| 80
~ 82
arithmetic compound assignment 66
assignment 50, 54
associativity 50
binary 50
bitwise 82, 86
boolean 74, 75, 78
cast 50
comparisons 74
compound assignment 79, 85

conditional 80, 90
decrement 69
dot 9
equality 75
execution order 52
extended assignments 90
floating-point 60
floating-point division 62
floating-point remainder 63
increment 69
`instanceof` 92
`instanceof` pattern match operator 276
`instanceof` type comparison operator 272
integer 82
integer arithmetic 59
integer division 62
integer remainder 62
logical 78
multiplicative 61
`new` *see* `new` **operator**
overflow 60
overloaded 59, 62
postfix 50
precedence 50
relational 74
shift 86
short-circuited 80
string concatenation 67
ternary 50
unary 50, 61
unary `-` 61
unary `+` 61
optimizations 5
optional annotation element 1566, 1570
`Optional` class 892, 942
absence of a value 942
creating 943
declaring and returning 942
querying 945
see also **numeric optional classes** 946

optional dependency 1181
optional operations 792
`OptionalDouble` 892, 946
`OptionalInt` 892, 946
`OptionalLong` 892, 946
optionally bounded collections 1487
or operator 83
Oracle CertView 1621
Oracle exam attempt 1621
Oracle JDK 3
Oracle MyLearn 1621
ordered collections 786, 803
ordered streams 893
    extracting lines from a `String` 904
    from a `BufferedReader` 904
    from a `CharSequence` 903
    from factory methods in the `Files` class 905
ordering mode 843
ordinal value 165, 295
OUT parameters 1534
outer instance 510, 511
`OutOfMemoryError` 376, 540
output 23, 1237
output stream 1237
`OutputStream` abstract class 1238
`OutputStreamWriter` class 1248
overflow 34, 60
overflow errors 483
overloaded 59
overloaded method resolution 267
overloaded methods
    ambiguous call 204
overloading 618
    constructors 112
    method resolution 267
    methods 108, 204
overloading vs. overriding 204
override criteria 619
override-equivalent 618, 623
overriding 392, 623

`@Override` 620, 1582
covariant return
`equals` 77, 746
`hashCode` 755
methods 204
override-equivalent 618
`toString` 68
overriding vs. overloading 204

P

package accessibility for class members 349, 353
package accessibility for top-level types 347
package directory 337
`package` statement 327, 330, 347
package-private accessibility for class members
see **package accessibility for class members**
package-private accessibility for top-level types
see **package accessibility for top-level types**
packages 328
classpath 341
declaration 327
definition 330
destination directory 337
hierarchy 328
`java.io` 1237
`java.lang` 427
`java.util` 785
location 341
members 328
package directory 337
root 341
running code from 339
shorthand 331
statement *see* **package statement**
unnamed 330
using 331
palindromes 690
parallel processing 1452
parallel streams 893, 1011

autoboxing 1018
behavioral parameters 912
benchmarking 1012
building parallel streams 936, 1011
collecting to a `ConcurrentMap` 985
computation-intensive operations 1013
concatenating 898
concurrent collectors 985, 991
data size 1013
determine if parallel 936
encounter order 955
execution 964, 968, 1012
factors affecting performance 1012
from collections 899
grouping to a `ConcurrentMap` 991
ordered 894
ordering 1017
parallel functional reduction 964
parallel mutable reduction 968
setting parallel execution mode 935, 936
shared state 1017
side effects 912, 950
non-interfering behaviors 1017
stateless behaviors 1017
splittable 1013
stateful operations 913, 914, 1017
unordered 894
parallelism 1372
parameter list *see* **formal parameters**
parameter passing
by value 128
variable arity 136
parameterized types 571, 609
as formal parameters 611
parameters 100
actual 128
array elements 134
effectively `final` 402
`final` 136, 234, 402
fixed arity 136

formal *see* **formal parameters**
implicit 106
main method 142
passing 127
primitives 129
program 142
references 131
`this` 106
variable arity 136, 137
parent class 193
parent directory (`..`) 1293
parent resource bundle 1112
parentheses 50
`ParseException` 373
parsing 1119
compact number form 1125
currency values 1123
date and time 1131
numbers 1123
percentage values 1124
using patterns 1155
parsing numeric values 436
partial application 725
partial implementation 220, 244
partly synchronizing a method 1395
pass by value 128
passing
parameters 127
references 131
variable arity parameter 136
passwords
reading from console 1260
path 1292
comparing with another path 1307
constructing relative path to another path 1304
converting 1301
converting to a real path 1306
converting to an absolute path 1302
creating 1294, 1295
interoperability with `File` class 1296

interoperability with `URI` class 1297
name elements 1292
name separator 1292, 1295
naming conventions 1292
normalizing 1303
querying 1298
resolving with another path 1303
`Path` interface 1291
see also **path**
paths
see **class path**
paths of execution 1371
path-separator character 342
pattern letters 1131, 1139
pattern match operator 276
pattern matching 276
pattern symbols 1129
pattern variable 277
cannot shadow local variable 278
declare final 278
scope 277, 278
shadow a field 278
patterns
format elements 1144
percentage 1120, 1121
formatting 1120, 1123
percentage formatter 1121, 1124
performance 5
period 1061
compared with duration 1076
creating 1061
date-based 1061, 1068
equality 1064
get methods 1063
immutable 1061
normalization 1065
parsing 1062
period-based loop 1066
plus/minus methods 1065
supported `ChronoUnit` s 1064

temporal arithmetic 1065
text representation 1062, 1071
thread-safe 1061
`with` methods 1064
`Period` class 1061
see **period**
periodic tasks 1445
`permits` clause 314
permitted direct subclasses 314
accessibility 316
contract 315
extending direct sealed superclass 315
`final` 315
locality 316
`non-sealed` 315
`sealed` 316
permitted direct subtypes
contract 318
enum types 318
record classes 318
permute elements 862
persistence
see **serialization**
plain data 301
plain data class 301
plain data objects 301
plain JAR 1193, 1210
Plain Old Java Objects (POJOs) 301
platform modules 1170
plurals 1151
poison value 1503
POJOs 301
polled locking 1466, 1471
polymorphic method 595
polymorphism 262, 280, 284, 520
polynomial 761
portability 4
Portable Operating System Interface (POSIX) 1329
positional order 646
position-based access 803

positive zero 61
POSIX file permissions
enum type 1330
full permissions 1347
group 1330
others 1330
owner 1330
read (r), write (w), execute (x) 1329
string format 1330
PosixFileAttributes interface 1334, 1336
PosixFileAttributeView interface 1338, 1340
PosixFilePermission enum type 1329
PosixFilePermissions class 1330
postfix operators 50
power 482
precedence rules 51
precision 56
predefined literals 31
predicate 691
Predicate functional interface 705
composing predicates 706
primitive type specializations 708
preempted threads 1390
preemptive scheduling 1390
prefix 33
0 33
0x 33
prepared statement 1530
substitutional parameterization 1530
primitive data types 41
see **primitive types**
primitive types 44
autoboxing 432
unboxing 434
see also **primitive values**
primitive values
assignment 54
binary representation 1242
equality 75
passing 129

principle exported package 1175
principle of least privilege (PoLP) 1615
print writers 1249
    formatted values 1251
    text representation of values 1250
printing values 23
`PrintWriter` class 1248
priorities 1389
priority
    priority queue 818
priority heap 790, 818
priority of thread 1389
    starvation 1416
priority queue 789, 790, 817
`PriorityBlockingQueue` 1500
`PriorityQueue` 817
`private` members 353
privileged code 1613
privileges 1613
PRNG
    see **pseudorandom number generators** 485
procedural programming paradigm 3
process of elimination 1623
process-based multitasking 1371
processes 1371
program
    application 19
    arguments 142
    command line 142
    compiling 20
    correctness 1389
    formatting 39
    performance 1389
    running 21
    single-file source-code 21
program arguments 142
program counter (PC) 1374
program output 23
programmatic locking 1464
    interruptible locking 1466, 1471

lock acquisition 1464
lock disciplines 1464
polled locking 1466, 1471
read-write locking 1470
reentrant 1469
reentrant locking 1465
timed locking 1466, 1471
unconditional locking 1471
programming to interfaces 649, 1200
proleptic year 1033
promotion 49
proper prefix 873, 874
properties 6
see also **class members**
properties file
comment 1105
escaping whitespace 1105
key-value pair syntax 1104
key-value pairs 1104
properties 1104
property list 1104
text file 1105
see also `Properties` **table**
see also **property resource file**
`Properties` table 1106
property
key-value pair 1104
see also **properties file**
see also `Properties` **table**
see also **property resource file**
property list 1104
see also **properties file**
see also **property resource file**
property resource file 1107
`.properties` file extension 1107
naming scheme 1107
properties 1107
see also **properties file**
see also **resource bundle**

PropertyResourceBundle 1109, 1112
protected members 352
provides-with directive 1182, 1204
pseudorandom generator 1426
pseudorandom number generators 902
see Math **class**
see Random **class**
pseudorandom numbers 484, 485
public 19
public API 1167, 1174, 1178
public members 352
punctuators 32

Q

qualified class instance creation expression 504, 505
qualified export 1180
see exports-to **directive**
qualified name 253, 494, 498, 499, 502, 503
qualified opens directive
see opens-to **directive**
qualified superclass constructor invocation 511
qualified this 506, 517
query results 1537
Queue 816
queue 787, 790
capacity-restricted 817
FIFO 823
head 816
LinkedList 817
PriorityQueue 817
Queue 816
unbounded 817
quotation mark 38, 39
quotient 62

R

race conditions 1017, 1393
radix

prefix 33
see **base** 433
ragged arrays 125
raised to power 482
random access by index 804
`Random` class 484, 1426
random numbers 484
random order 862
range
character values 42
floating-point values 42
integer values 42
range of values for date fields 1033
range of values for time fields 1032
range-view 658, 803, 812, 813, 848, 1490
ranking criteria 646
raw types 573, 577, 862
unchecked warnings 577
read lock 1470
read operation 543
readability 1177
readable 1174
readers 1245
Read-Evaluate-Print Loop (REPL) 23
reading binary values 1242
reading bytes 1238
reading characters 1253, 1254
reading data 1237
`ReadLock` 1470
read-write lock
fairness policy 1474
read lock 1470
reentrant 1474
write lock 1470
read-write locking 1470
realization 246
reclaiming memory 535
record classes 170, 301, 302, 884
augmenting 306
basic syntax 302

compact canonical record constructor 306, 308
component fields 304
component list 302
generating methods of the `Object` class 305
generic 313
get methods 302, 304
immutable 304
implementing interfaces 312
implicit canonical record constructor 304
local 496, 521
member declarations 309
nested 313, 494
normal canonical record constructor 306, 307
normal non-canonical record constructor 307, 309
permitted direct subtypes 318
permitted in sealed interfaces 313
record construction 305
record elements 302
record header 302
record name 302
record serialization 312
restrictions 306
serialization 1267
shallowly immutable 304
static member 496
top-level 496
record components 302
record header 302
record name 302
recursive type bound 593, 615
`RecursiveAction` 1452
`RecursiveTask` 1452
reduced graph 1171
reducing complexity 326
reduction operations 949
reentrant 1469
reentrant locking 1465
fairness policy 1465
reentrant synchronization 1394
`ReentrantLock` 1465

reference types 99, 102, 197, 493
classes 99
enum types 289, 290
interface types 239, 248
reference values 8
reference variables 102
references 8, 11, 102, 113, 127
abstract types 223
aliases 77, 131
array 118, 121, 262
assignment 55
casting 49, 271
dangling 535
downcasting 45
dynamic type 198
equality 76
field 536
local 536
narrowing conversions 45
`null` *see* **null reference**
parameterized types 571
passing 131
raw types 590
reachable 535, 536
static type 198
`super` 208, 517
`this` 106, 517
upcasting 45
widening conversions 45
reflection 631, 1169, 1182
Reflection API 1591
reflective access 1195
reifiable types 625
relational databases 1516
relational model 1516
relational operators 74
relational tables
columns 1516
constraints 1516
rows 1516

relative adjusters 1044
relative path 1292
reliability 5
reliable configuration 1167
remainder 62
remainder operator
% 62
remove whitespace 455
repeatable annotation type 1580
replacing characters 454
required annotation element 1566
requires directive 1173, 1174, 1181
readability 1177
requires static directive 1181
requires transitive directive 1179, 1181
reserved keywords 31
reserved literals
false 32, 37
null *see* **null reference**
true 32, 37
resizable arrays 789, 790, 804
resource
see **resource bundle**
resource bundle 1106
base name 1106
candidate bundle name 1112
default resource bundle 1106, 1107, 1113, 1114
fallback 1114
key-based lookup 1108
key-value pairs 1106
locating resources 1111
naming convention 1106
parent resource bundle 1112
property resource file 1107
resource 1106
resource bundle class 1107
result bundle 1112
resource bundle class 1107
resource bundle family 1106
see also **resource bundle**

resource declaration statement 412
resource leakage 409
resource variable
    final 412
ResourceBundle 1106, 1107
resources 409, 538
restricted type name var 100
result bundle 1112
result set 1537
    concurrency 1543, 1544, 1545
    concurrency mode 1545
    cursor 1538
    customizing 1543
    extracting column values 1540
    fetch size 1540
    holdability 1543, 1544, 1545
    holdable 1544
    metadata 1547
    navigational direction 1543, 1544
    scrollable 1543
    type 1543, 1544
    updatability 1543, 1544
retaining elements 793
retention policy values for @Retention 1572
rethrowing exceptions 403
return statement 186
    @return tag 186
return type
    covariant
return value 10
reuse of code 193, 197
reverse DNS 329, 1175
reverse-order key set 849
reverse-order view 849
rhyming order 773
right associativity 52
rightmost bit 34
ripple effect 197
robustness 5, 418, 1611
role relationship 197

root
see **inheritance hierarchy**
root component 1291
root node 1291
`round` method 481
rounding functions 480
rounding mode 1126
rounding numbers 481
round-robin scheduling 1390
rows 1516
`Runnable` functional interface 1374, 1375, 1380, 1428
`RUNNABLE` state 1386
Running 1207
running a Java application 21
running from a modular JAR 1195
running from a runtime image 1209
runtime
bounds checking 121
runtime checks 48, 650
`Runtime` class 427, 539
runtime class 428
runtime constant pool 1374
runtime data areas 1373
runtime environment 535, 1104
runtime image 3, 1208
runtime stack
see **JVM stack**
runtime type
see **dynamic type**
`RuntimeException` 374, 399

S

scheduled executor service
scheduling one-shot tasks 1444
scheduling periodic tasks 1445
scheduled thread pool 1431
`ScheduledExecutorService` 1444
schedulers 1390
schema 1297

scientific notation 36
scope 354
block 356
`catch` clause 383, 412
class 354
disjoint 357
scope of local variables 326
sealed classes 314
direct sealed superclass 315
in compilation unit 319
`permits` clause 314
permitted direct subclasses 314, 315
sealed interfaces
direct sealed superinterface 317
in compilation unit 319
`permits` clause 317
permitted direct subtypes 317
searching 785, 858
arrays 868
collections 863
searching in string 453
searching in string builder 468
secure 5
secure coding 1604
security guidelines 1614
security policies 1612
security threats 1606
`SecurityManager` class 427
seed 486
`SELECT` statement 1519
selection statements 154
selector expression 162
self-reference 147
self-referential data structure 568
semantic definition 30
semicolon 101
sensitive information leakage 1607
separators 32, 51
sequence 786
sequential I/O 1237

sequential streams 893
execution 960
extracting lines from a `String` 904
from a `BufferedReader` 904
from a `CharSequence` 903
from arrays 900
from collections 899
from factory methods in the `Files` class 905
ordered 893
unordered 893
serial access 1482
serialization 1265
customizing 1272
inheritance 1274
`serialVersionUID` 1279
transient fields 1271
versioning 1277
`serialVersionUID` 1279
service 1182, 1200
service consumer 1182, 1201, 1206
service interface 1200, 1201
service loader 1201, 1204
service locator 1182, 1201, 1204
service provider 1182, 1201, 1203
service provider interface
see **service interface**
`ServiceLoader` class 1201
`ServiceProvider.Provider` interface 1204
services 326
`Set` 806
set 756, 786
capacity 809
`HashSet` 808
`LinkedHashSet` 808
load factor 809
`NavigableSet` 812
`Set` 806
`SortedSet` 812
`TreeSet` 814
unmodifiable 806, 807

unmodifiable views 858
set methods 360
set operation 588
setter methods *see* **set methods**
SHA-256 digest algorithm 1607
shadowing fields by local variables 115
shadowing local variables 516
shadowing members 687
shallow copying 428
shallowly immutable 304
shared field 1457
shared memory 1373
shift
compound operators 90
distance 86
left 86
operators 86
shifting 86
`short` 32, 42
short cut
see **symbolic link**
short-circuit 80
evaluation 80
short-circuit execution 910, 949, 954
shuffle elements 862
sign bit 86
sign fill 86
signature 108, 204, 618
signed shift 86
simple
assignment operator 54
`if` 154
statement 101
simple name 347
simple type name 328
`SimpleFormat` 1119
simplicity 4
single implementation inheritance 195
single inheritance of implementation 195
single quote (') 37, 38

single source file 22
single static import 333
single thread executor 1430
single thread scheduled executor 1431
single-element annotation 1567, 1569
    array-valued single-element annotation 1570
    optional annotation element 1570
    single-element array-valued single-element annotation 1570
    `value()` element 1569
single-element annotation type 1564
single-file source-code program 21
single-line comment 7, 40
singleton 1260
single-valued maps 832
sink nodes 1176
skeletal source file 327
skip list 1496
sleeping 1399
software security 1604
sorted map 787
    comparator 847
sorted set 786, 812, 813
    comparator 813
    half-open interval 812
`SortedMap` 847
`SortedSet` 812
sorting 785, 858, 931
sorting arrays 867
sorting lists 860
source
    file 20, 330
    file name 327
    file structure 327
source directory 1223
source of data 1237
SP *see* **space**
space 38
spaces 39
spawning threads 1382
special character values 38

specialization 13
specialized classes 196
spurious wakeup 1403
SQL 1518
SQL injection 1529, 1609
SQL statement API
    basic statement 1527
    callable statements 1534
    prepared statement 1530
SQL statements 1526
SQLException 373, 1522
square root 482
stability guarantee 913, 932
stack
    generic 600
    LIFO deque 823
stack frame
    see **method execution**
stack trace 369, 372, 417
    see **method execution**
StackOverflowError 376
stacks 817
standard error stream 372, 1259
standard input stream 1259
standard modules 1170
    JMOD archive 1170
standard out 24
standard output stream 1259
standard time 1086
StandardCopyOption enum type 1312
starting threads 1375
starvation 1416
state *see* **object state**
stateful operations 910
stateless behavioral parameters 912
stateless operations 910
statement block 684
statement terminator 1518
statements 101
    break 182

compound 101
conditional 154
continue 184
control flow 101
control transfer 181
declaration 65, 70, 81, 101
do-while 175
empty 101
expression 70, 101
for(;;) 176
for(:) 178
if 154
if-else 155
iteration 174
labeled 181
resource declaration 412
return 186
selection 154
simple 101
simple if 154
switch statement 157, 162
synchronized 1395, 1396
throw 388
transfer 181
try 379
try -with-resources 409
while 174
yield 166
static
members *see* **static members**
methods 11, 13, 101
variable initialization 103
variables *see* **static variables**
static 10, 253, 333
static code
see **static context** 99, 100
static context 99, 100
static factory method 361, 1120
static field initializers 99
static fields 13, 113

static import 333
conflicts 336
on demand 333
shadow static members 335
single static import 333
static initializer blocks 99, 547
declaration order 547
exception handling 551
local variable type inference 547, 549
`static` keyword 113, 547
static fields 113
static methods 115
static local enum types 494, 496
static local interfaces 494, 496
static local nested types
see **static local types** 493
static local record classes 494, 496
static local types 493
enum types 494, 496
interfaces 494, 496
record classes 494, 496
see **local enum types**
see **local interfaces**
see **local record class**
static member classes 496, 497
class file name 499
declaration 497
direct access 500
fully qualified name 499
instantiation 499
qualified name 498
UML class diagram 500
static member enum types 496
static member interfaces 496
static member record classes 496
static member types 493
static member class 493
static member enum type 494
static member interface 493

static member record class 494
`static` members 10, 11, 13
static method references 728
static methods 115
hiding from superclass 205
static nested classes
see **static member types**
static nested enum types
see **static member enum types**
static nested interfaces
see **static member interfaces**
static nested record class
see **static member record class**
static nested types
see **static member types**
static type 198
`static` variables 11, 13, 106
stored functions 1534
IN parameter 1534
INOUT parameter 1536
OUT parameter 1534
stored procedures 1534
IN parameter 1534
INOUT parameter 1536
OUT parameter 1534
stream 800
transformations 907
Stream API 891
`BaseStream` interface 891
building streams 892
`Collector` interface 892
`Collectors` 892
`DoubleStream` interface 891
`IntStream` interface 891
`LongStream` interface 891
`Stream` interface 891
`Stream` interface 891
stream mapping 908
stream operations 883

non-interfering behavioral parameters 912
stateless behavioral parameters 912
stream pipelines
composing 888
executing 889
streams 883
see also **I/O streams**
aggregate operations 883
building from a `BufferedReader` 904
building from a `CharSequence` 903
building from a range 900
building from arrays 900
building from collections 899
building from directory entries 1349
building from specified values 895
building from the `Random` class 902
building using factory methods in the `Files` class 905
collecting into a list 974
collecting into an array 973
compare with collections 890
concatenating 897
conversion 936
counting elements 955
data source 886
dropping elements 912, 915
empty 895
encounter order 893
examining elements 922
execution mode 935
extracting lines from a `String` 904
filtering 912
finding first/any 954
finding min/max 956
finite 894
flattening 926
functional reduction 955, 956, 957, 966, 974
infinite 894
intermediate operations 887, 907, 910
lazy execution 909
mapping 907, 923

map-reduce transformations 907
matching operations 951
method chaining 908
mutable reduction 949, 966
see also **collectors**
numeric 887, 894
numeric summary statistics 974
object 887, 894
operations with side effects 948, 950
ordered 893
parallel streams 893, 964, 968, 1011
reduction operations 949, 966
replacing each element with multiple elements 929
searching operations 949
selecting distinct elements 917
sequential 893
sequential execution 960
setting as unordered 934
short-circuit execution 910, 949, 954
skipping elements 917
sorting 931
stability guarantee 913, 932
stateful operations 910
stateless operations 910
Stream API 891
stream mapping 908
stream operations 883
stream pipelines 888
taking elements 912, 915
terminal stream operations 887, 948
truncating 919
unordered 893
`strictfp` 58
string builders 69
appending 469
capacity 466, 472
`compareTo` 469
construction 467
copying characters 467
deleting 470

extracting substrings 468
individual characters 467
inserting 470
joining 452
length 467
mutable 466
reading characters 467
replacing 470
reversing 471
searching 468
setting length 472
substrings 468
trimming 472
`String` class
see **strings**
string conversion 46, 68, 457
string literal pool 442
interned 442
string literals 442
`case` labels 164
hash value 164
interned 442
string pattern 1129
`StringBuffer` legacy class
thread-safe 466
`StringBuilder` class
see **string builders**
strings
capacity 472
changing case 450
compact 442
`compareTo` 449
comparing 446, 449, 763
concatenation 67, 450
concatenation operator `+` 69
contains 454
conversions 457
convert to character array 447
copying characters 447
creating 442

empty 443, 446
ends with 454
equals 449
extracting lines 904
extracting substrings 455
finding index 453
formatted 459
ignoring case in comparison 449
immutable 442
indenting lines 458
individual characters 447, 467
initializing 442
integer stream 446
internal representation 441
interned 442, 444
joining 452, 845, 986
length 446, 447
lexicographical ordering 449
literals 39, 442
natural order 763
read character at index 446, 447
repeating 451
replacing 454
searching 453
selector expression 158
starts with 454
streams 903, 904
string literal pool 442
stripping 455
substrings 455
text blocks 460
thread-safe 442
transforming 457
trimming 455
strongly typed language 48
Structured Query Language 1518
style-based formatter for dates and times 1135
subclass 13, 15, 193
subdirectories 1291
subformat pattern 1150

subinterface 246
sublist 658
submap view 847, 848
sub-maps 1495
subsequence 446
subset 806, 810, 812, 813, 1490
subsignature 619
substitutional parameterization 1530
substring searching 453, 468
substrings 453, 455, 468
subtags 1102
subtype bivariance 583
subtype contravariance 583
subtype covariance 261, 262, 582, 583
subtype invariance 583
subtype notation `<:` 586
subtype relationship 650
subtypes 244, 280, 583, 594
subtype-supertype relationship 45, 197
suffix
    `D` 36
    `F` 36
    `L` 32
`super` 250
    construct 213
    keyword 203, 208
    reference 208
superclass 13, 193
superclass constructor 110
superinterfaces 246
superset 810
supertypes 244, 280, 584
`Supplier` functional interface 701
    primitive type specializations 702
suppressed exceptions 372, 417
suppressed warnings
    deprecation warnings 1587
    unchecked warnings 1587
switch block
    local variable scope 171

switch rules 162
switch expression 166
arrow notation 168
case labels 166
colon notation 166
comparison with switch statement 171
default clause 168
exhaustive 167, 168, 169
fall-through 167
local variable scope 171
return multiple values 170
yield statement 166, 168
switch rules 162
mutually exclusive 163
switch statement 157
arrow notation 162
break 159
case labels 158
colon notation 157, 159
comparison with switch expression 171
default clause 158, 163
enum types 165
execution 158
exhaustive 165
fall-through 158
local variable scope 171
selector expression 162
using strings 164
switching between threads 1372
symbolic link 1293, 1308
synchronization 1391
synchronized
keyword 1392
methods 1392
starvation 1416
statements 1395, 1396
vs. volatile 1460
synchronized block 1395
synchronized collections 1481
compound mutually exclusive operations 1484

serial access 1482
underlying collection 1481
synchronized method 1392
synchronized views of collections
see **synchronized collections**
`SynchronousQueue` 1500
syntactically legal 30
`System` 427
`out` 24
reassigning standard streams 1259
standard streams 1259
system clock 1034
system default zone ID 1078
system modules 1216

T

TAB *see* **horizontal tab**
table definition 1517
tabs 39
tabulators 39
tagging interface
see **marker interfaces**
tags 40
tail map view 847, 848
tail set view 812, 813, 1490
taking stream elements 912, 915
target directory 1191
target parallelism level 1430
target reference 726
target type 572, 684
target typing 686
task 1371, 1427
task cancellation 1448
task execution policy 1430
TCL 1518
telephone directory order 449
temporal arithmetic 1044, 1058, 1065, 1073
temporal fields 1031
temporal objects

access specific temporal values 1036, 1081
before/after methods 1038
combining date and time 1034
common methods 1030
comparing 1038
conversion to `Date` 1092
creating 1031, 1079
date 1028
date fields 1028
date units 1044
date-time 1028
fields 1028, 1029
formatting 1131
`get` methods 1036, 1081
immutable 1028, 1079
method naming convention 1030
parsing 1131
plus/minus methods 1044, 1085
range of values for date fields 1033
range of values for time fields 1032
temporal arithmetic 1044, 1065, 1066, 1075, 1084
temporal fields 1031, 1040
thread-safe 1028, 1079
time 1028
time fields 1028
time units 1044
`with` methods 1039, 1082
zoned date-time 1028, 1078
`TemporalAmount` interface 1065
see also `Duration`
see also `Period`
`TemporalField` interface
see also `ChronoField`
`TemporalUnit` interface
see also `ChronoUnit`
terminal stream operations 887, 948
averaging 976
collecting into a list 974
collecting into an array 973

counting elements 955
eager execution 889
finding first/any 949, 954
finding min/max 956
functional reduction 949, 955, 956, 957, 974
accumulator 958, 959, 964
combiner 958, 964
identity value 958, 959, 964
parallel execution 964
sequential execution 960
matching operations 949, 951
mutable reduction 949, 966, 973, 974
accumulator 967
combiner 967
parallel stream execution 968
sequential stream execution 967
supplier 967
see also **collectors**
operations with side effects 948, 950
reduction operations 949, 966
see also **collectors**
searching operations 949
short-circuit operations 949, 954
summary statistics 974, 976
summation 975
`TERMINATED` state 1386, 1409
terminating loops 182
ternary conditional expressions
see also **conditional expressions** 90
ternary conditional operator 50, 90
text block 460
closing delimiter 460
content 460
escape sequences 461
essential whitespace 464
incidental whitespace 464
indentation 464
opening delimiter 460
text files
reading from 1253

stream to read lines 1350
writing to 1251
text representation 428
this
reference 106
this keyword
qualified form 506
with new 505
this() constructor call 211, 557
thousands separator 1129
Thread class 1374
thread coordination 1400
thread execution
runtime organization 1373
thread interference 1377, 1393
thread interruption rule 1419
thread lifecycle 1384
thread of execution 1371
thread pool 1427, 1429
thread safety 1372
Thread start rule 1418
Thread termination rule 1418
thread-based multitasking 1371
thread-local variable 1457
ThreadLocalRandom 1426
threads 5, 427, 536, 1406
acquire lock 1392
alive 1381, 1387
BLOCKED state 1386
blocking operation 1409
child 1377, 1382
class 1374
code executed 1375
controlling termination 1409
creation 1374
current thread 1376
daemon 1381
deadlocks 1412
death 369
deprecated methods 1412

determining state 1385
entry set for 1384
exception propagation 369
extending `Thread` 1379
fairness 1412
happens-before relationship 1418
`IllegalThreadStateException` 1382
interrupt status 1397
interrupted 1403
`InterruptedException` 1397
joining 1387, 1406
JVM stack 536
live 536
livelock 1414
liveness 1412
main thread 1377, 1382
names 1376
`NEW` state 1386, 1390
non-observable substate 1386
non-runnable state 1387
normal 1381
notification 1400, 1402
notifying 429, 1400
Object lock rule 1418
priorities 1389
priority 1387
`run()` method 1376
`Runnable` 1374, 1375, 1380
`RUNNABLE` state 1386
running 1379
running thread 1387
scheduler 1389
shared field 1457
shared memory 1373
sleeping 1387, 1399
spawning 1382
spurious wakeup 1403
`start()` method 1376
starting 1375, 1379, 1390
starvation 1416

states 1385
switching 1372
synchronization 1391
`synchronized` 1392
synchronized method 1392
tasks 1427
`TERMINATED` state 1386, 1409
termination 1409
`Thread` class 1374
thread coordination 1400
Thread interruption rule 1419
thread lifecycle 1384
Thread start rule 1418
thread status 1382
Thread termination rule 1418
thread-local 1457
`TIMED_WAITING` state 1386
transitions 1385
Volatile field rule 1418, 1458
wait set for 1401
waiting 429, 1400
`WAITING` state 1386
yielding 1387, 1390
thread-safe 466, 647, 791, 1028, 1079
immutable classes 358
thread-safe code 1391, 1455
thread-safe collection 1481
thread-safety 358
atomic variables 1460
immutability 1457
intrinsic locking 1464
programmatic locking 1464
synchronized collections 1481
unmodifiable collections 1481
volatile fields 1457
`throw` statement 388
`Throwable` 370, 407, 427
throw-and-catch paradigm 367
throwing exceptions 367
`throws` clause 390

and rethrowing 404
time fields 1028
time gap 1086
time overlap 1086
time standard 1077
time units 1044, 1425
time zone 1028
local time 1077
see zone ID
time zone offset 1077
time zone rules 1078, 1086, 1088
timed locking 1466, 1471
`TIMED_WAITING` state 1386
timeline 1028
instant 1078
local 1078
time-sliced scheduling 1390
timestamps 1053
`TimeUnit` 1425
`ToDoubleBiFunction` interface 720
`ToDoubleFunction` interface 714
`ToIntBiFunction` interface 720
`ToIntFunction` interface 714
tokens 30
`ToLongBiFunction` interface 720
`ToLongFunction` interface 714
top-level classes 493, 496
top-level enum types 493, 496
top-level interfaces 493, 496
top-level record classes 493, 496
top-level types 493
`toString` method 428, 433, 434
total order 763
total ordering 867
Transaction Control Language (TCL) 1518
transactions 1549
auto-commit mode 1549
commit 1549
rollback 1549
savepoint 1549

transfer statements 181
TransferQueue 1500, 1502
transforming strings 457
transient fields 1271
transitive relation 196
transitive relationship 1418
tree 1291
    traversal 1351
tree map 849
tree set 814, 849
TreeMap 849
trees 790
TreeSet 814
trim method 455
true literal 32, 37
truth-values 37, 43
try block 379
try-catch-finally construct 377
try-with-resources 147, 409, 1523
    basic 412
    concise syntax 413
    extended 412
    header 412
    var in header 412
tuples 1516
two's complement 33, 34, 86
two-arity 697
type annotation 1576
type cast 48
type cast expression 271
type checking 684
type comparison operator 272
type contexts 1576
type declarations 327
    nested types 493
    top-level types 493, 496
type erasure 578, 615, 624
type hierarchy 45, 197
type import
    see import

type inference 143
type information 567
type instance 571
type parameter constraints 593
type parameters 635
actual 571
bounds 593
formal 569
recursive bounds 613
restrictions 570
type pattern 277
type variables 569
type witness 596, 597, 930
types
arrays 581
boolean 41, 43
byte 32, 42
casting 48
char 42
classes *see* **classes**
comparison 272
compatibility 48
double 42
exceptions 370
float 42
floating-point 41, 42
int 32, 42
integers 41
integral types 41
long 32, 42
numeric types 41
parsing 436
short 32, 42
unrelated 572
wrappers 431
see also **classes**
type-safe 266
type-safety 648, 650
type Value method 434, 436

U

UML 6
accessibility 350
aggregation 16
classes 6
inheritance 14
see also **diagrams**
UML class diagram
non-static member classes 510
static member classes 500
UML notation for members 350
UN M.49 three-digit area code 1100
unary arithmetic operators 61
unary numeric promotion 49
unary operators 50
`UnaryOperator` interface 722
composing unary operators 722
primitive type specializations 723
unbounded collections 1487
unbounded instance method references 731
unbounded type parameter 583, 585
unbounded type references 592
unbounded wildcard
`?` 583, 584
unbounded wildcard references 590
unboxing 434
`do-while` statement 175
`for(;;)` statement 176
`for(:)` statement 179
`if` statement 154
`if-else` statement 155
`switch` statement 158
`while` statement 175
unboxing conversions 45
uncaught exceptions 369
unchecked 629, 632
unchecked array creation warning 632
unchecked call warning 578, 590
unchecked cast warning 627, 629

unchecked conversion warning 573, 578, 587, 629, 648
unchecked conversions 46
unchecked exceptions 376
unchecked generic array creation warning 632, 652, 661, 807, 835, 865
unchecked warnings 45, 577, 578, 1587
    `@SuppressWarnings("unchecked")` 629
    call warning 578, 590
    cast warning 627
    conversion warning 578
    generic array creation warning 632
    heap pollution warning 632, 1589
    unchecked generic array warning 1589
unconditional locking 1471
underflow 34, 61
underlying collection 1481
uni-`catch` clause 378, 399
    effectively `final` parameter 402
Unicode 37, 42, 439, 449, 1245
Unicode character encoding 1105
Unicode locale/language extension 1122
unidirectional edges 1170
Unified Modeling Language *see* **UML**
Uniform Resource Identifier (URI) 1294
union 806
union of alternatives 401
unmodifiable 662
unmodifiable collections 651, 807, 859, 1481
unmodifiable lists 651, 662, 803
unmodifiable maps 834
unmodifiable sets 807
unmodifiable view of a collection 858, 859
unmodifiable view of a list 858
unmodifiable view of a map 859
unmodifiable view of a navigable map 859
unmodifiable view of a navigable set 858
unmodifiable view of a set 858
unmodifiable view of a sorted map 859
unmodifiable view of a sorted set 858
unmodifiable views 858, 859
unmodifiable views of collections 1481

unnamed module 316, 1210
unnamed package 316
unordered streams 893
from the `Random` class 902
unqualified export 1182
see **`exports`** directive
unreachable code 383, 403
unsafe casts 273
unsigned integer 437
unsupported operations 792, 833
`UnsupportedOperationException` 375, 796, 799
`UnsupportedTemporalTypeException` 1037, 1066, 1072, 1074, 1079, 1135
unused keywords 31
upcasting 45
updatability 1543
update expression 176
`UPDATE` statement 1519
upper bound 583, 593
upper bounded wildcard
`? extends T` 583
upper bounded wildcard references 591
`URI` class 1294, 1297
schema 1297
user threads
see **normal thread**
`uses` directive 1182, 1206
using arrays 121
using packages 331
using variables 102
UTC (Coordinated Universal Time) 1077
UTC/Greenwich 1077
`Z` 1077
UTF-16 encoding scheme 441, 442
utility class 115, 253, 362

V

value collection 833
value obfuscation 1607
value objects 787

`value()` element 1564, 1565, 1569
`valueOf` method 432, 433, 457
values 34, 832
    constants 32
    overflow 34
    underflow 34
    wrap-around 34
    see also **variables**
`var` 573
`var` declaration
    see **local variable type inference**
varargs
    variable arity 136
varargs method
    see **variable arity method**
varargs parameter
    see **variable arity parameter**
variable arity argument 661, 865
variable arity call 140
variable arity method 136
variable arity parameter 137, 632, 865, 1308
    generic array creation 632
    heap pollution 632
variable capture 689
variable declarations 102, 356
variable initialization 11, 102, 104
variables 8, 102
    blank final 136
    blank `final` local 233, 234
    constant values 231
    default values 103
    effectively `final` 689
    `final` 231
    identifiers 102
    in interfaces 256
    initialization *see* **variable initialization**
    lifetime 106
    local 143, 356
    parameters 100, 128

reference variable 102
references 102
static 11
var-type inferred parameters 682
Vector 804
vector 804
Venn diagrams 793
versioning 1277
view of a list 658, 662
viewing dependencies 1218
class-level dependencies 1220
module-level dependencies 1220
package-level dependencies 1218
views 812, 841
virtual method invocation
see **dynamic method lookup**
Virtual Private Network (VPN) 1605
VirtualMachineError 376
visibility 1458
visibility modifiers
see **access modifiers**
void 19, 186, 431
void return 683
Void wrapper class 431
volatile
vs. synchronized 1460
Volatile field rule 1418, 1458
volatile fields 1457
visibility 1458
volume name 1291

W

wait set 1400, 1401
starvation 1416
waiting 1400
timeout 1403
WAITING state 1386
waiting threads 429
WHERE clause 1519, 1520

`while` statement 174
whitespace 39, 455, 456
whole-part relationship 196
widening conversions
    primitive 44
    references 45
widening reference conversions 198, 272, 586
wider range 44
wildcard parameterized types 586
    type hierarchy 583, 584, 585
wildcard types 582
    see **wildcards**
wildcards 581, 582
    `?` 590
    `? extends T` 591
    `? super T` 591
    assignment 586
    bounded 583
    capture 606
    capture conversion 608
    nested 609
    parameterized types 586
    restrictions 586
    subtype relationships 581
    type hierarchy 583, 584, 585
withers 1039
work stealing pool 1430
    target parallelism level 1430
worker threads 1427
wrapper 1237
wrapper classes 41, 427, 428, 431, 746, 756, 763
    interned values 435
wrapper type 45, 46, 59, 70
write lock 1470
write operation 544
`WriteLock` 1470
writers 1245
writing binary values 1242
writing bytes 1238
writing characters 1242, 1253, 1257

writing data 1237

X

xor 83

Y

`yield` statement 166, 168
yielding 1390

Z

`Z` 1077
zero
    negative 61
    positive 61
zero fill shift 86
zero-arity 697
zero-based index 646, 803
zone ID 1077
    available zone IDs 1077
    system default 1078
zoned date-time
    accessing time zone 1081
    accessing time zone offset 1081
    conversion to `Date` 1092
    converting to different time zone 1083
    converting to instant 1082
    see also **daylight saving time (DST)**
    see **temporal objects**
`ZonedDateTime` class 1076, 1078
    see **temporal objects**
    see **zoned date-time**
`ZoneId` class 1076
`ZoneOffset` class 1076

Previous chapter
Appendix G. Java Logging API Overview

Next chapter
OCP Oracle Certified Professional Java SE 21 Developer (Exam 1Z0-830) Java SE 17 Developer (Exam 1Z0-829) Programmers Guide